

VDBFuzz: Understanding and Detecting Crash Bugs in Vector Database Management Systems

Shenao Wang^{*†}
shenao.wang@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Zhao Liu^{*}
r3pwnx@gmail.com
360 AI Security Lab
Beijing, China

Yanjie Zhao[†]
yanjie_zhao@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

Quanchen Zou[‡]
zouquanchen@gmail.com
360 AI Security Lab
Beijing, China

Haoyu Wang^{†‡}
haoyu.wang@hust.edu.cn
Huazhong University of Science and
Technology
Wuhan, China

ABSTRACT

Vector Database Management Systems (VDBMSs) have become critical in LLM-integrated applications. However, their inherent complexity, including high-dimensional data structures, diverse indexing strategies, and heterogeneous implementations, makes them prone to reliability issues. Among these, crash bugs caused by boundary condition failures, such as invalid configurations and mismatched data dimensions, are particularly severe. These bugs can result in serious consequences like data loss, corrupted indexes, and cascading failures. To address this gap, we propose VDBFuzz, the first fuzzing framework specifically designed to detect VDBMS crash bugs through boundary value testing. VDBFuzz systematically leverages techniques to collect high-quality seeds, generate edge-case inputs, and explore complex API interactions. We evaluated VDBFuzz on 8 representative VDBMSs, including native systems (e.g., Weaviate, Milvus), libraries (e.g., Faiss, hnswlib), and extended systems (e.g., pgvector, sqlite-vec). VDBFuzz achieved up to 3x higher code coverage compared to state-of-the-art tools such as RESTler and Schemathesis, uncovering 19 previously unknown bugs, including 13 crash vulnerabilities and 6 runtime exceptions. These results highlight VDBFuzz’s effectiveness in improving the robustness and reliability of VDBMSs.

CCS CONCEPTS

• **Security and privacy** → **Software security engineering**: *Denial-of-service attacks*; • **Software and its engineering** → **Software testing and debugging**.

^{*}Both authors contributed equally to this research.

[†]Hubei Key Laboratory of Distributed System Security, Hubei Engineering Research Center on Big Data Security, School of Cyber Science and Engineering, Huazhong University of Science and Technology.

[‡]Haoyu Wang (haoyu.wang@hust.edu.cn) and Quanchen Zou (zouquanchen@gmail.com) are the corresponding authors.

ACM Reference Format:

Shenao Wang, Zhao Liu, Yanjie Zhao, Quanchen Zou, and Haoyu Wang. 2026. VDBFuzz: Understanding and Detecting Crash Bugs in Vector Database Management Systems. In *2026 IEEE/ACM 48th International Conference on Software Engineering (ICSE '26)*, April 12–18, 2026, Rio de Janeiro, Brazil. ACM, New York, NY, USA, 12 pages. <https://doi.org/10.1145/3744916.3773139>

1 INTRODUCTION

Vector Database Management Systems (VDBMSs) have emerged as a crucial technology in the era of Large Language Models (LLMs). Unlike traditional relational or NoSQL databases [22], which primarily handle structured or semi-structured data, VDBMSs are specifically optimized for storing and querying vector embeddings, which are numerical representations of multimodal data such as text, images, and audio [41, 53]. Popular frameworks such as Pinecone [43], Milvus [35], and Weaviate [61] have become indispensable in powering various applications, including search engines [53], recommendation systems [53, 56], and LLM-based workflows [26, 70]. In particular, VDBMSs play a critical role in enabling advanced LLM capabilities, such as Retrieval-Augmented Generation (RAG) [26] and long-term memory [24, 25]. As LLMs are increasingly integrated into real-world applications, VDBMSs have become the backbone for supporting their scalability and effectiveness.

Despite their growing adoption and importance, ensuring the reliability and robustness of VDBMS remains a significant challenge [58, 64]. These systems are inherently complex due to their reliance on high-dimensional data structures, various indexing strategies, and heterogeneous language implementations [35, 46, 61]. The complexity makes them particularly susceptible to reliability issues [58, 64], such as crashes, unexpected errors, or degraded performance. According to a recent empirical study [64], crash bugs account for 15.1% of all observed defects in VDBMS, frequently manifesting as abrupt process terminations, segmentation faults, or unrecoverable runtime exceptions. For example, systems like Qdrant [46] and txtai [39] exhibit high crash rates (up to 40.7% of their total bugs). These failures can lead to severe consequences such as data loss, corrupted vector indexes, or cascading failures in downstream applications, emphasizing the urgent need for systematic methods to detect and mitigate crash bugs in VDBMSs.



VDBFuzz: 理解并检测向量数据库管理系统中的崩溃漏洞

王世诺*†shenaowang@hust.edu.cn

华中科技大学, 中国武汉

赵刘*†r3pwnx@gmail.com

360人工智能安全实验室, 中国北京

闫杰 Zhao†yanjie_zhao@hust.edu.cn

华中科技大学
中国武汉

Quanchen Zou†
zouquanchen@gmail.com
360人工智能安全实验室, 中国北京

霍宇 Wang†haoyuwang@hust.edu.cn

华中科技大学 武汉, 中国

摘要

向量数据库管理系统 (VDBMS) 在集成LLM的应用中已变得至关重要。然而, 其固有的复杂性——包括高维数据结构、多样化的索引策略以及异构实现方式——使其极易出现可靠性问题。其中, 由边界条件故障 (如无效配置或数据维度不匹配) 引发的崩溃错误尤为严重。这类错误可能导致数据丢失、索引损坏以及连锁故障等严重后果。为解决这一问题, 我们提出了VDBFuzz——首个专门通过边界值测试检测 VDBMS 崩溃错误的模糊测试框架。VDBFuzz系统性地运用多种技术手段来采集高质量测试种子、生成边缘场景输入并探索复杂的API交互关系。我们在8种代表性VDBMS上对VDBFuzz进行了评估, 涵盖原生系统 (如Weaviate、Milvus)、库类工具 (如Faiss、hnsplib) 以及扩展系统 (如pgvector、sqlite-vec)。与RESTler和Schemathesis等前沿工具相比, VDBFuzz实现了高达3倍的代码覆盖率, 发现了19个此前未知的漏洞, 其中包括13个崩溃漏洞和6个运行时异常。这些结果充分证明了VDBFuzz在提升数据库管理系统稳健性和可靠性方面的卓越效果。

中国化学会概念

• 安全性与隐私保护→软件安全工程; 拒绝服务攻击; • 软件及其工程设计→软件测试与调试。

*两位作者对本研究的贡献均等。

† 湖北分布式系统安全重点实验室、湖北大数据安全工程研究中心、华中科技大学网络科学与工程学院。

‡ 王浩宇 (haoyuwang@hust.edu.cn) 与邹全晨 (zouquanchen@gmail.com) 为通讯作者。

请使用 nonacm 选项或 ACM Engage 类来启用 CC 许可证  本作品采用知识共享署名-非商业性使用4.0国际许可协议授权。

ICSE 2026 年4月12日至18日, 巴西里约热内卢

© 2026 版权所有/作者所有。

ACM ISBN 979-8-4007-2025-3/26/04

<https://doi.org/10.1145/3744916.3773139>

ACM参考格式:

Shenao Wang、Zhao Liu、Yanjie Zhao、Quanchen Zou 和 Haoyu Wang. 2026. VDBFuzz: 理解和检测向量数据库管理系统中的崩溃漏洞。收录于2026年IEEE/ACM第48届国际软件工程大会 (ICSE '26), 2026年4月12-18日, 巴西里约热内卢。ACM出版社, 美国纽约州纽约市, 12页。 <https://doi.org/10.1145/3744916.3773139>

1 引言

向量数据库管理系统 (VDBMS) 已成为大型语言模型 (LLM) 时代的一项关键技术。与主要处理结构化或半结构化数据的传统关系型或NoSQL数据库[22]不同, VDBMS专门针对存储和查询向量嵌入进行优化——这些向量嵌入是文本、图像和音频等多模态数据的数值表示[41, 53]。Pinecone[43]、Milvus[35]和Weaviate[61]等主流框架已成为驱动各类应用不可或缺的核心组件, 包括搜索引擎[53]、推荐系统[53, 56]以及基于LLM的工作流[26, 70]。尤其值得注意的是, VDBMS在实现检索增强生成 (RAG) [26]和长时记忆[24, 25]等先进LLM功能方面发挥着关键作用。随着LLM日益广泛地应用于实际场景, VDBMS已成为支撑其可扩展性与高效性的核心基础。

尽管 VDBMS 系统的应用日益广泛且重要性不断提升, 但确保其可靠性和稳健性仍是一项重大挑战[58, 64]。这些系统本质上非常复杂, 这源于其对高维数据结构、多种索引策略以及异构语言实现的依赖[35, 46, 61]。这种复杂性使其特别容易出现可靠性问题[58, 64], 如崩溃、意外错误或性能下降。根据最近的一项实证研究[64], 崩溃错误占 VDBMS 所有已观测缺陷的15.1%, 通常表现为进程突然终止、段错误或不可恢复的运行时异常。例如, Qdrant[46]和txtai[39]等系统的崩溃率极高 (高达总错误数的40.7%)。此类故障可能导致严重后果, 如数据丢失、向量索引损坏或下游应用程序发生级联故障, 这凸显了亟需建立系统化方法来检测和缓解VDBMS中的崩溃错误。

However, existing works on VDBMS testing have largely focused on performance evaluation [4, 38, 45, 71], with limited emphasis on systematic reliability testing. Performance testing primarily measures efficiency under limited predefined scenarios, but it fails to uncover deeper issues that require exploring diverse and unpredictable inputs beyond predefined ones. The most relevant work METMAP has introduced metamorphic testing for detecting false vector matching problems in LLM augmented generation. Nevertheless, it focuses on algorithm-level inaccuracies in vector similarity computations rather than addressing implementation-specific defects in VDBMSs. While traditional testing approaches, such as unit tests [15, 60] and integration tests [34], are commonly used to ensure the functionality of VDBMSs, they primarily rely on manually written test cases by developers and often fail to uncover deeper reliability issues and edge-case vulnerabilities [17, 59]. Fuzz testing has proven to be an effective approach for uncovering hidden bugs and vulnerabilities in other domains, such as traditional DBMS [19, 23, 53, 67], operating systems [7, 10, 68], and other various applications [8, 11, 65]. Despite these advancements, there is still a significant gap in applying fuzz testing to VDBMSs, particularly for detecting implementation-level reliability issues.

To address these gaps, we conducted a preliminary study to analyze the characteristics of crash bugs and developed VDBFuzz, the first fuzzing framework specifically designed for VDBMSs. Our preliminary study of crash bugs across 15 state-of-the-art VDBMSs revealed that 44.5% of these defects arise from improper handling of boundary inputs (e.g., extreme parameter values, mismatched data dimensions, or malformed structures) and 15.6% of them arise from dependencies on specific API call sequences (e.g., incorrect order, omission, or redundancy of operations). Based on these insights, VDBFuzz systematically explores crash-inducing behaviors through three key stages: 1) *Seed Collection*, which extracts representative API usage patterns from unit tests, example scripts, and documentation to form a structured test corpus; 2) *Template-based Input Mutation*, which generates diverse inputs by applying reusable mutation patterns targeting boundary values, invalid configurations, and edge cases; and 3) *API Sequence Mutation*, which evaluates the robustness of VDBMSs by mutating API call sequences to uncover defects caused by logical inconsistencies or invalid state transitions. By combining these components, VDBFuzz is capable of systematically testing VDBMSs and detecting VDBMS crash bugs.

To evaluate the effectiveness of VDBFuzz, we conducted comprehensive experiments on 8 representative VDBMSs, including native VDBMSs (e.g., Weaviate, Milvus, Qdrant), VDBMS libraries (e.g., Faiss, HNSWLib, Annoy), and extended VDBMSs (e.g., Pgvector, SQLite-vec). When compared to state-of-the-art fuzzing tools such as RESTler and Schemathesis, VDBFuzz demonstrated superior performance on native VDBMSs, achieving 3x higher coverage on average. For example, in Weaviate, VDBFuzz covered 15,485 lines of code, compared to RESTler's 4,519 and Schemathesis's 1,811, over the same 120-minute testing period. An ablation study further confirmed the complementary nature of VDBFuzz's mutation strategies, with each component contributing meaningfully to the overall coverage. Overall, VDBFuzz uncovered 19 previously unknown bugs across all tested systems, including 13 crash vulnerabilities and 6 runtime exceptions, with root causes linked to boundary

condition failures such as dimension mismatches, invalid configurations, and malformed inputs. These results validate VDBFuzz's effectiveness in systematically detecting crash bugs, significantly improving the robustness and reliability of VDBMSs.

Contributions. To summarize, this paper makes the following key contributions:

- **Investigation of Crash Bugs in VDBMSs.** We conducted a detailed analysis of crash bugs across 15 VDBMSs, revealing that 44.5% of defects arise from improper handling of boundary inputs, while 15.6% stem from invalid API call sequences. These findings informed the design of VDBFuzz to systematically address these critical vulnerabilities.
- **Design and Implementation of VDBFuzz:** We propose VDBFuzz, the first fuzzing framework specifically designed for VDBMSs. VDBFuzz systematically explores crash-inducing behaviors by analyzing real-world API usage, generating diverse edge-case inputs, and testing complex API interactions to ensure robustness and reliability.
- **Real-world Impact:** We evaluated VDBFuzz on 8 representative VDBMSs. VDBFuzz achieved 3x higher code coverage compared to state-of-the-art fuzzing tools, uncovering 19 previously unknown bugs, including 13 crash vulnerabilities and 6 runtime exceptions.

Artifact Availability. We have publicly released the full source code of VDBFuzz at <https://github.com/security-pride/VDBFuzz>.

2 BACKGROUND AND MOTIVATION

Current VDBMS. VDBMSs are specialized databases designed to handle high-dimensional vector embeddings. A typical VDBMS architecture separates the server and client components to enable scalability and flexibility [41, 58]. The server handles the core functionalities, such as vector storage, index construction, and query processing, while the client provides interfaces and wrappers for interacting with the server. This separation allows VDBMSs to support multiple programming languages and frameworks, enabling developers to integrate them easily into diverse applications. As shown in Table 1, VDBMSs can be categorized into three main types based on their design and functionality.

Table 1: Comparison of Popular VDBMS by Category.

Category	VDBMS	Star	Fork	Server	Client
Library	Faiss [16]	37.3k	4.1k	C++	Python
	Hnswlib [40]	4.9k	731	C++	Python
	Annoy [51]	14k	1.2k	C++	Python
Native	Weaviate [61]	14.7k	1.1k	Go	Py/JS/Java/Go
	Milvus [35]	37.7k	3.4k	Go	Py/JS/Java/Go
	Qdrant [46]	26.2k	1.8k	Rust	Py/JS/Java/Rust/Go
Extended	pgvector [44]	17.7k	896	C	Py/JS/Java/Go
	sqlite-vec [52]	6.2k	233	C	Python

(1) **Vector Libraries:** These libraries are typically foundational implementations of indexing or search algorithms, designed to provide high-performance vector operations. Examples include Faiss [16],

然而，现有的 VDBMS 测试研究主要集中在性能评估 [4, 38, 45, 71] 方面，对系统性可靠性测试的关注较为有限。性能测试主要在预设的有限场景下衡量效率，但难以揭示需要探索超出预设范围的多样化及不可预测输入所带来的深层问题。最具代表性的研究 MeTMaP 提出了元变形测试方法，用于检测大语言模型增强生成中的虚假向量匹配问题；不过该方法侧重于向量相似度计算中的算法层面误差，而非解决 VDBMS 系统的具体实现缺陷。虽然单元测试 [15, 60] 和集成测试 [34] 等传统测试方法常被用于验证 VDBMS 的功能性，但这些方法主要依赖开发人员手动编写测试用例，往往无法发现深层次的可靠性问题或边缘场景漏洞 [17, 59]。模糊测试已被证明是发现其他领域中隐藏缺陷和漏洞的有效方法，例如传统数据库管理系统 [19, 23, 53, 67]、操作系统 [7, 10, 68] 以及其他各类应用程序 [8, 11, 65]。尽管已有这些进展，但在将模糊测试应用于虚拟数据库管理系统（VDBMS）时仍存在显著差距，尤其是在检测实现层面的可靠性问题方面。

为弥补这些不足，我们开展了一项初步研究以分析崩溃漏洞的特征，并开发了 VDBFuzz——首个专为关系型数据库管理系统（VDBMS）设计的模糊测试框架。通过对 15 款先进 VDBMS 中崩溃漏洞的初步分析发现：44.5% 的缺陷源于对边界输入的不当处理（如极端参数值、数据维度不匹配或结构异常）；另有 15.6% 的缺陷则源自对特定 API 调用序列的依赖（如操作顺序错误、步骤遗漏或重复执行）。基于这些分析结果，VDBFuzz 通过三个关键阶段系统性地探索导致崩溃的行为：1）种子集收集——从单元测试、示例脚本和文档中提取具有代表性的 API 使用模式，构建结构化的测试数据集；2）基于模板的输入变异——通过应用可复用的变异模式（针对边界值、无效配置及边缘情况）生成多样化输入；3）API 调用序列变异——通过改变 API 调用顺序来评估数据库管理系统（VDBMS）的健壮性，从而发现由逻辑不一致或状态转换异常引发的缺陷。综合这些模块，VDBFuzz 能够系统化地测试数据库管理系统并检测 VDBMS 崩溃错误。

为评估 VDBFuzz 的检测效能，我们针对 8 种代表性数据库管理系统（VDBMS）开展了全面实验，涵盖原生数据库系统（如 Weaviate、Milvus、Qdrant）、VDBMS 库（如 Faiss、HNSWLib、Annoy）以及扩展型数据库系统（如 Pgvector、Sqlite-vec）。与 RESTler 和 Schemathesis 等前沿模糊测试工具相比，VDBFuzz 在原生数据库系统中展现出更优性能，平均覆盖率高出三倍。以 Weaviate 为例，在相同的 120 分钟测试期间，VDBFuzz 覆盖了 15,485 行代码，远超 RESTler 的 4,519 行和 Schemathesis 的 1,811 行。消融实验进一步证实了 VDBFuzz 各变异策略的互补性——每个组件均对整体覆盖率产生显著贡献。总体而言，VDBFuzz 在所有测试系统中共发现 19 个未知漏洞，

包括 13 个崩溃漏洞和 6 个运行时异常，其根本原因均与边界条件失效相关（如维度不匹配、配置无效或输入格式错误）。这些结果充分验证了 VDBFuzz 在系统化检测崩溃漏洞方面的有效性，显著提升了数据库管理系统的稳健性和可靠性。贡献。总之，本文主要做出以下关键贡献：

- **对 VDBMS 系统崩溃漏洞的研究。** 我们对 15 种 VDBMS 系统的崩溃漏洞进行了详细分析，发现 44.5% 的缺陷源于边界输入处理不当，而 15.6% 则源自无效的 API 调用序列。这些发现为 VDBFuzz 的设计提供了依据，使其能够系统性地应对这些关键漏洞。
- **VDBFuzz 的设计与实现：** 我们提出了 VDBFuzz——首个专为数据库管理系统（VDBMS）设计的模糊测试框架。该框架通过分析实际 API 使用场景、生成多样化的边缘情况输入以及测试复杂的 API 交互，系统性地探索导致系统崩溃的行为，从而确保测试结果的稳健性和可靠性。
- **实际应用效果：** 我们在 8 种代表性 VDB 管理系统上对 VDBFuzz 进行了评估。其代码覆盖率较最先进的模糊测试工具高出 3 倍，共发现 19 个先前未知的漏洞，其中包括 13 个崩溃漏洞和 6 个运行时异常。

工具可获取性。 我们已将 VDBFuzz 的完整源代码公开发布于 <http://github.com/security-pride/VDBFuzz>。

2 背景与动机

当前 VDBMS。 VDBMS 是专为处理高维向量嵌入而设计的专用数据库。典型的 VDBMS 架构将服务器与客户端组件分离，以实现可扩展性和灵活性 [41, 58]。服务器负责处理核心功能（如向量存储、索引构建和查询处理），而客户端则提供与服务器交互所需的接口和封装层。这种分离结构使 VDBMS 能够支持多种编程语言和框架，便于开发者将其轻松集成到各类应用中。如表 1 所示，根据设计和功能特性，VDBMS 可分为三大主要类型。

表1：按类别划分的主流 VDBMS 对比。

类别	VDBMS	星	餐叉	服务器	客户
图书馆	法伊斯 [16]	37.3k	4.1k	C++	巨蛇
	Hnswlib [40]	4.9k	731	C++	巨蛇
	令人烦恼 [51]	14k	1.2k	C++	巨蛇
本地的	Weaviate [61]	14.7k	1.1k	出发	Py/JS/Java/Go
	米尔沃斯 [35]	37.7k	3.4k	出发	Py/JS/Java/Go
	Qdrant [46]	26.2k	1.8k	铁锈	Python/JavaScript/Java/Rust/Go
扩展版	pgvector [44]	17.7k	896	C	Py/JS/Java/Go
	sqlite-vec [52]	6.2k	233	C	巨蛇

(1) **向量库：** 这类库通常是索引或搜索算法的基础实现方案，旨在提供高性能的向量运算功能。典型代表包括 Faiss [16]、Hnswlib [40] 和 Annoy [51]。它们作为轻量级库直接嵌入应

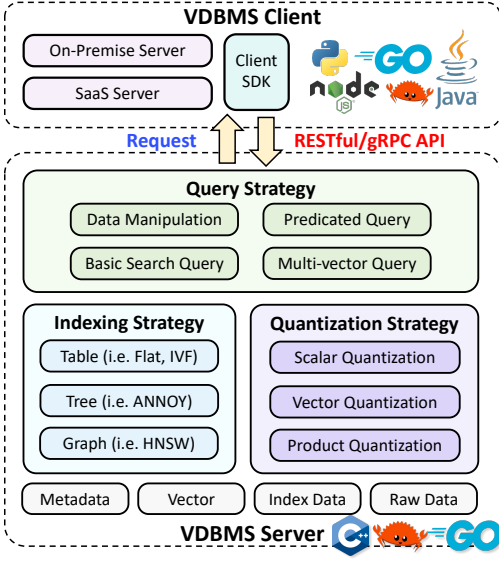


Figure 1: Typical architecture of VDBMSs.

Hnswlib [40], and Annoy [51]. They operate as lightweight libraries embedded directly into applications and are implemented in high-performance languages like C++, with bindings available for other languages such as Python. While vector libraries excel in speed and efficiency, they often lack the complete functionality of a VDBMS, such as user access control and distributed deployment.

(2) **Native VDBMS:** These are purpose-built systems optimized to handle vector workloads natively, providing a complete suite of functionalities for vector similarity search, including dynamic index updates, metadata filtering, and multi-vector queries [41, 53]. Examples of native VDBMS include Milvus [35], Qdrant [46], and Weaviate [61]. These systems typically expose their functionalities via network APIs, such as REST or gRPC, enabling distributed deployment and large-scale production use cases.

(3) **Extended Databases with Vector Support:** Extended databases are traditional databases, such as relational or NoSQL systems, that incorporate additional functionality to support vector similarity searches [9, 69]. These systems typically rely on plugins, extensions, or external libraries to perform vector-related tasks. Examples include PostgreSQL [44] with the pgvector extension [42], SQLite [52] with vector similarity search [3], and MongoDB with integrated vector search capabilities [36]. These systems allow organizations to leverage existing database infrastructure while adding vector similarity search functionality [66].

VDBMS Workflow. The architecture of a VDBMS is designed to efficiently handle the entire lifecycle of vector data, from storage to indexing and query execution. A typical workflow within a VDBMS involves three key steps, as illustrated in Figure 1. First, vector data, typically represented as dense fixed-length floating-point arrays, is stored alongside metadata such as labels and timestamps to enable hybrid queries. Next, the system constructs indexes to accelerate similarity searches [63]. Popular indexing techniques include table-based indexes (e.g. LSH [6, 31] and IVF [18]), tree-based approaches (e.g., ANNOY [51]), graph-based methods (e.g., HNSW [32]), and

quantization techniques (e.g., PQ [18, 28]). Finally, during query execution, the system retrieves approximate nearest neighbors from the index and applies metadata filters to refine results. VDBMSs support various distance metrics, such as Euclidean distance and cosine similarity, and some systems, like Milvus and Qdrant, allow multi-vector queries to aggregate results dynamically.

VDBMS Bugs. While VDBMSs provide efficient solutions for managing high-dimensional vector embeddings, they are not immune to implementation flaws, which can lead to severe system vulnerabilities. VDBMS bugs refer to defects in the implementation of vector storage, indexing, or query execution processes that may cause system crashes or incorrect behaviors. For instance, a notable example is a bug in the Faiss similarity search library’s IndexIVFPQFastScan implementation¹. As shown in Listing 1, when the nlist parameter (number of Voronoi cells) is not byte-aligned, attempting to reconstruct vectors via reconstruct or reconstruct_batch triggers out-of-bounds memory access due to invalid decoding of internal codes.

```
1 index = faiss.IndexIVFPQFastScan(
2     faiss.IndexFlatL2(dim),
3     dim,
4     nlist=100,
5     dim // 2,
6     4,
7     faiss.METRIC_L2
8 )
9 index.train(vec)
```

Listing 1: A crash bug in Faiss IndexIVFPQFastScan caused by non-byte-aligned nlist values.

3 A PRELIMINARY STUDY

To gain a deeper understanding of crash bugs in VDBMSs, we conducted a preliminary study to investigate the root cause of these defects. This study aims to identify the functions where crash bugs originate, the characteristics of crash-inducing inputs, and any dependencies on specific function call sequences, thereby providing actionable insights for the design of VDBFuzz.

3.1 Study Overview

To systematically investigate the root causes of crash bugs in VDBMS, we design a methodology that includes data collection, filtering, and classification of crash-related defects.

Data Collection and Scope. We based our analysis on the dataset from Xie et al. [64], which includes 1,671 bug-fix pull requests (PRs) from 15 widely used open-source VDBMSs, including Faiss [16], Milvus [35], and Qdrant [46]. Using this dataset, we extracted and analyzed 247 crash-related PRs, which account for 15.1% of all recorded defects. To ensure the relevance of our study, we manually reviewed these PRs to confirm their association with crash symptoms, such as process termination, segmentation faults, or unrecoverable runtime exceptions. To ensure the quality of our dataset, we applied the following filtering criteria: 1) The PR must document a root cause and include sufficient details, such as the triggering input or the affected function; 2) The PR must address an

¹<https://github.com/facebookresearch/faiss/issues/4089>

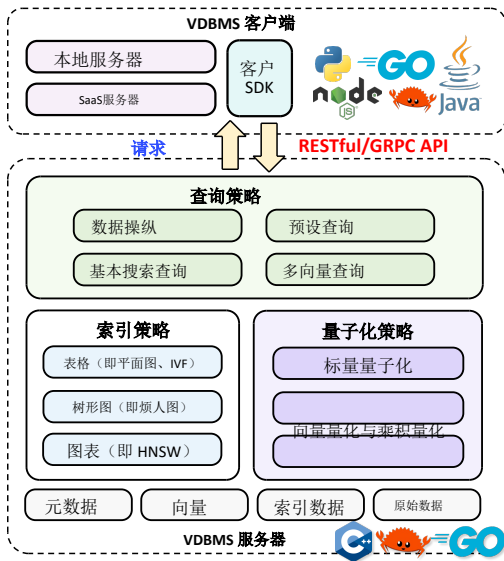


图1: VDBMS的典型架构。

用程序中, 采用C++等高性能语言编写, 并提供适用于Python等其他语言的接口。虽然向量库在速度和效率方面表现优异, 但往往缺乏VDBMS的完整功能(如用户访问控制和分布式部署)。

(2) **原生VDBMS**: 这类系统是专为原生处理向量工作负载而设计的优化方案, 提供完整的向量相似性搜索功能集, 包括动态索引更新、元数据过滤和多向量查询[41, 53]。典型的原生VDBMS包括Milvus[35]、Qdrant[46]和Weaviate[61]。这些系统通常通过REST或gRPC等网络API接口提供功能, 支持分布式部署及大规模生产场景的应用。

(3) **支持向量搜索的扩展数据库**: 扩展数据库是传统数据库(如关系型或NoSQL系统)通过添加额外功能来实现向量相似性搜索[9, 69]。这类系统通常依赖插件、扩展模块或外部库来执行与向量相关的任务, 例如PostgreSQL[44]中的pgvector扩展模块[42]、Sqlite[52]提供的向量相似性搜索功能[3], 以及具备集成向量搜索能力的MongoDB[36]。这些系统使组织能够在利用现有数据库基础设施的同时, 新增向量相似性搜索功能[66]。

VDBMS 工作流程。VDBMS的架构设计旨在高效处理矢量数据的整个生命周期, 涵盖从存储到索引构建及查询执行的全过程。如图1所示, VDBMS中的典型工作流程包含三个关键步骤: 首先, 矢量数据(通常以密集固定长度浮点数组形式存储)与标签、时间戳等元数据一同存储, 以支持混合查询; 其次, 系统构建索引以加速相似性搜索[63]; 常用索引技术包括基于表格的索引(如LSH[6, 31]和IVF[18])、基于树结构的方法(如annoy[51])、基于图结构的方法(如HNSW[32])以

及量化技术(如PQ[18, 28]); 最后, 在查询执行时, 系统从索引中检索近似最近邻节点并应用元数据过滤器来优化结果。矢量数据库管理系统支持多种距离度量方式(如欧几里得距离和余弦相似度), 部分系统(如Milvus和Qdrant)还支持动态聚合多向量查询结果。

VDBMS 漏洞。虽然向量数据库管理系统(VDBMS)为管理高维向量嵌入提供了高效解决方案, 但它们并非完全不受实现缺陷的影响, 这些缺陷可能导致严重的系统漏洞。VDBMS漏洞指的是向量存储、索引或查询执行过程中的实现缺陷, 可能引发系统崩溃或异常行为。例如, Faiss相似性搜索库中的IndexIVFPQFastScan实现中就存在一个典型漏洞¹。<如清单1所示, 当nlist参数(沃罗诺伊单元数量)未进行字节对齐时, 尝试通过reconstruct或reconstruct_batch函数重建向量会因内部编解码错误而触发超出内存范围的访问操作。

```
1 索引 = fai ss. 索引 IVF PQ 快速扫描 (fai
2
3  ss)。索引 平坦 L2 (维度), 维度, n
4
5 列表 = 100, 维度 // 2、4
6
7 执行 SS. 公制_L2)
8
9 索引. 训练集 (向量)
```

列表 1: 由非字节对齐的nlist值引起的Faiss Index IVFPQFastScan程序崩溃错误。

3 一项初步研究

为深入理解VDBMS中的崩溃漏洞, 我们开展了一项初步研究以探究这些缺陷的根本原因。本研究旨在明确崩溃漏洞产生的函数、引发崩溃的输入特征以及对特定函数调用序列的依赖关系, 从而为VDBFuzz的设计提供切实可行的见解。

3.1 研究概述

为系统性地探究VDBMS中崩溃错误的根本原因, 我们设计了一套涵盖崩溃相关缺陷数据收集、筛选与分类的分析方法。

数据收集与范围。我们的分析基于Xie等人的数据集。[64]该数据集包含来自15种广泛使用的开源版本数据库管理系统(VDBMS)的1,671个漏洞修复拉取请求(PR), 包括Faiss[16]、Milvus[35]和Qdrant[46]。基于此数据集, 我们提取并分析了247个与崩溃相关的PR, 占有记录缺陷总数的15.1%。为确保研究相关性, 我们人工审阅了这些PR以确认其与崩溃症状(如进程终止、段错误或不可恢复的运行时异常)存在关联; 为保证数据集质量, 我们采用了以下筛选标准: 1) PR必须记录根本原因并包含充分细节(如触发输入或受影响函数); 2) PR必须涉及

¹<https://github.com/facebookresearch/faiss/issues/4089>

Table 2: The number of collected VDBMS crash bugs.

VDBMS	Ori.	Filt.	Storage	Index	Query	Final
Chroma [12]	15	9	3	2	1	6
DeepLake [1]	18	11	1	–	6	7
Faiss [16]	1	–	–	1	–	1
hnswlib [40]	2	–	–	1	1	2
LanceDB [29]	13	7	2	1	3	6
Marqo [33]	8	2	1	–	5	6
Milvus [35]	62	17	9	3	33	45
pgvecto [42]	15	5	1	2	7	10
Qdrant [46]	11	4	5	1	1	7
txtai [39]	2	1	–	–	1	1
Usearch [54]	8	4	–	1	3	4
Vespa [55]	23	12	–	–	11	11
Voyager [2]	2	–	–	–	2	2
Weaviate [61]	26	6	10	2	8	20
Total	206	78	32	14	82	128

actual crash bug, confirmed by its resolution and subsequent merging into the main branch of the repository; 3) PRs with ambiguous descriptions or bugs unrelated to core VDBMS functionalities (e.g., storage, indexing, or query execution) were excluded. After filtering out ambiguous or irrelevant cases, we obtained a refined dataset of 128 confirmed crash bugs. Our analysis focused on three key aspects: the affected functions, the characteristics of their inputs, and their dependency on specific function call sequences.

Threats to Validity. Similar to other empirical studies, our results have the following limitations, which should be taken into account. 1) *Potential Bias.* Our analysis relies on the dataset from Xie et al. [64], which is composed of bug-fix pull requests from 15 open-source VDBMSs. This dataset may not fully represent all crash bugs encountered in other proprietary or less widely adopted VDBMSs. Additionally, the quality and completeness of the PR descriptions vary across projects, potentially impacting the accuracy of our classification. 2) *Filtering Criteria.* The filtering criteria we applied to identify crash-related bugs were designed to focus on core functionalities (storage, indexing, and query execution). However, this may have excluded other relevant crash bugs, such as those arising from deployment, configuration, or build processes, which might also impact system reliability in real-world scenarios. 3) *Manual Review and Labeling.* Although we manually reviewed and labeled the crash bugs, human error and subjectivity may have influenced the classification. Despite these limitations, the preliminary study provides a foundational understanding of crash bugs in VDBMSs and highlights areas for further research and tool development.

3.2 General Findings

Our analysis of 128 crash bugs across 15 VDBMSs reveals three complementary perspectives on their root causes: (1) *where* the crashes occur (i.e., affected functional modules), (2) *how* improper inputs trigger crashes, and (3) *when* specific operation sequences lead to failures. To make these relations explicit, Table 3 summarizes how the 128 crashes are partitioned by primary root cause category. In particular, 56 crashes are mainly triggered by improper inputs, and are further analyzed in Table 4; 20 crashes are mainly caused

Table 3: Overview of root causes for the 128 analyzed crash bugs.

Root Cause Category	# Bugs	Percentage (%)
Input-related Crashes	56	43.8
Operation-sequence Dependencies	20	15.6
Data Races	11	8.6
High-concurrency Issues	6	4.7
Dependency Issues	4	3.1
Internal System Errors	31	24.2
Total	128	-

Table 4: Crash-inducing input categories with examples.

Input	Examples	Frequency (%)
Null/Zero Values	Zero-length vectors	21 (37.5%)
Out-of-Bounds Values	Negative indices	7 (12.5%)
Type Mismatches	Mismatched data types	8 (14.3%)
Size Constraints	Vector size beyond limits	3 (5.4%)
Index/Query Constraints	Invalid search parameters	17 (30.4%)
Total	–	56 (100.0%)

by operation-sequence dependencies, and are detailed in Table 5. The remaining crashes are attributed to other causes, such as data races, high-concurrency issues, dependency problems, and internal system errors that are not directly exposed at the user-input level.

Functions Affected by Crash Bugs. Our analysis shows that crash bugs are primarily concentrated in three core functions: storage, indexing, and query execution. Query execution functions are the most affected, accounting for 64.1% of the total crash bugs. These often involve complex logic for handling vector search queries, ranking, and filtering, making them particularly prone to defects. Storage-related functions, responsible for persisting and retrieving vector data, account for 25% of the crashes. Indexing-related functions, which manage the creation and maintenance of vector indices, contribute to 10.9% of the crash bugs.

Crash-Inducing Inputs. As shown in Table 4, crash-inducing inputs can be classified into five primary categories: null or zero values, excessively large or negative values, type mismatches, violations of size constraints, and configuration constraints. Null or zero values caused the most crashes (21 out of 56, 37.5%), often leading to invalid states or unexpected behaviors in query execution or storage operations. Out-of-bounds values, such as excessively large or negative inputs (e.g., passing a vector of unsupported dimensions or negative indices), accounted for 12.5% of crashes, frequently resulting in memory-related issues like buffer overflows or segmentation faults. Type mismatches, where the input type did not match the expected type (e.g., providing a string where a numeric vector ID was required), contributed to 14.3% of crashes, exposing weaknesses in type validation mechanisms. Violations of size constraints, such as inputs exceeding the maximum allowed dimensions for vectors or batch sizes, led to 5.4% of crashes. Configuration constraints, specific to indexing or query API parameters (e.g., incorrectly configuring the dimensionality of an index or the number of nearest neighbors for a search query), accounted for 30.4% of crashes.

表2：已收集的 VDBMS 崩溃错误数量。

VDBMS	Ori.	Filt.	存储	索引	查询	最后
色度 [12]	15	9	3	2	1	6
DeepLake [1]	18	11	1	–	6	7
法伊斯 [16]	1	–	–	1	–	1
hnsplib [40]	2	–	–	1	1	2
LanceDB [29]	13	7	2	1	3	6
Marqo [33]	8	2	1	–	5	6
米尔沃斯 [35]	62	17	9	3	33	45
pgvecto [42]	15	5	1	2	7	10
Qdrant [46]	11	4	5	1	1	7
txtai [39]	2	1	–	–	1	1
Usearch [54]	8	4	–	1	3	4
维斯帕 [55]	23	12	–	–	11	11
旅行者号 [2]	2	–	–	–	2	2
Weaviate [61]	26	6	10	2	8	20
总计	206	78	32	14	82	128

1) 经修复并最终合并至仓库主分支后确认的实际崩溃错误；
2) 描述含糊或与核心 VDBMS 功能（如存储、索引或查询执行）无关的错误提交（PR）均被排除。经过筛选后，我们获得了一个包含128个已确认崩溃错误的精炼数据集。分析重点聚焦于三个关键维度：受影响函数、输入特征及其对特定函数调用序列的依赖性。

效度威胁。与其他实证研究类似，本研究结果存在以下局限性，需予以考虑。

1) *潜在偏倚。*我们的分析基于Xie等人的数据集。[64]该数据集由来自15种开源关系型数据库管理系统（VDBMS）的漏洞修复拉取请求组成。该数据集可能无法完全涵盖其他专有或使用较少的VDBMS中遇到的所有崩溃错误。此外，不同项目中提交请求描述的质量和完整性存在差异，这可能影响我们分类结果的准确性。2) *筛选标准：*我们用于识别崩溃相关错误的筛选标准旨在聚焦核心功能（存储、索引构建及查询执行）。但这可能导致某些相关的崩溃错误被遗漏，例如由部署、配置或构建流程引发的错误——这些错误同样可能影响实际应用场景中的系统可靠性。3) *人工审核与标注：*尽管我们对崩溃错误进行了人工审核和标注，但人为误差和主观判断仍可能影响分类结果。尽管存在这些局限性，本初步研究仍为理解VDBMS中的崩溃错误奠定了基础，并指出了未来研究与工具开发的重点方向。

3.2 总体发现

我们对15种VDBMS中128个崩溃错误的分析揭示了关于其根本原因的三种互补视角：(1) *崩溃发生的位置*（即受影响的功能模块）；(2) *不当输入如何引发崩溃*；(3) *特定操作序列何时导致故障*。为明确这些关系，表3总结了这128个崩溃事件按主要根本原因类别划分的情况。具体而言，56个崩溃主要由不当输入触发，详见表4；另有20个崩溃主要由

表3：分析的128个崩溃错误的根本原因概述。

根本原因分类	#错误	百分比 (%)
与输入相关的崩溃事件	56	43.8
操作序列依赖关系	20	15.6
数据竞赛	11	8.6
高并发问题	6	4.7
依赖问题	4	3.1
内部系统错误	31	24.2
总计	128	–

表4：导致碰撞的输入类别及示例。

输入；输入内容	示例	频率 (%)
空值/零值	零长度向量	21 (37.5%)
超出范围的数值	负指数	7 (12.5%)
类型不匹配	不匹配的数据类型	8 (14.3%)
尺寸限制	向量大小超出限制	3 (5.4%)
索引/查询约束	搜索参数无效	17 (30.4%)
总计	–	56 (100.0%)

这些故障主要由操作序列依赖性引起，具体细节详见表5。其余崩溃事件则归因于其他原因，如数据竞争、高并发问题、依赖关系问题以及未在用户输入层面直接暴露的系统内部错误。

受崩溃漏洞影响的功能。我们的分析表明，崩溃漏洞主要集中在三个核心功能：存储、索引和查询执行。其中查询执行功能受影响最为严重，占总崩溃漏洞的64.1%，这些功能通常涉及处理向量搜索查询、排序和过滤等复杂逻辑，因此特别容易出现缺陷；负责存储和检索向量数据的存储相关功能占崩溃案例的25%；而管理向量索引创建与维护的索引相关功能则贡献了10.9%的崩溃漏洞。

导致崩溃的输入。如表4所示，导致崩溃的输入可分为五大类：空值或零值、过大或负值、类型不匹配、尺寸约束违规以及配置约束问题。空值或零值引发的崩溃最多（56例中的21例，占37.5%），常导致查询执行或存储操作中出现无效状态或异常行为；超出范围的输入值（如过大的数值或负索引值，例如传递非支持维度的向量）占崩溃案例的12.5%，常引发缓冲区溢出或段错误等内存相关问题；类型不匹配（输入类型与预期类型不符，例如在要求数值型向量ID处传入字符串）占崩溃案例的14.3%，暴露出类型验证机制的缺陷；尺寸约束违规（如输入值超过向量或批量处理的最大允许维度）占崩溃案例的5.4%；而针对索引或查询API参数特有的配置约束问题（例如（错误配置索引维度或搜索查询中最近邻数量）占故障总数的30.4%。

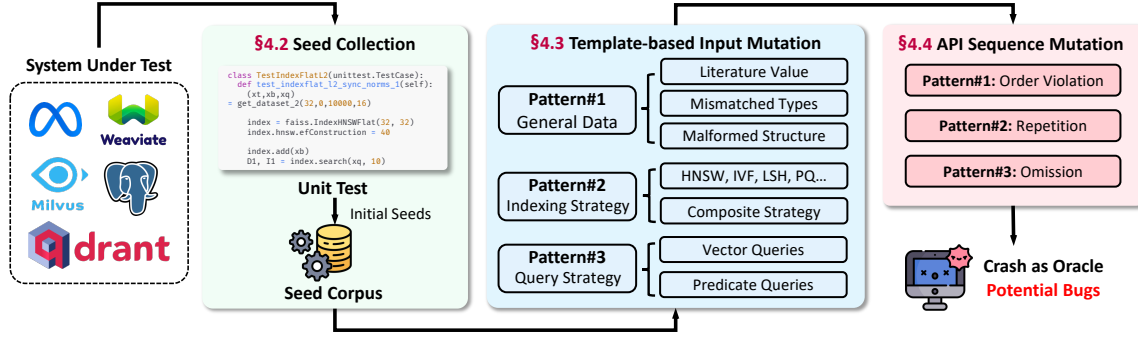


Figure 2: The overall workflow of VDBFuzz.

Table 5: Crash bugs caused by operation dependencies.

Dependency Issue	Frequency	Percentage (%)
Missing Critical Operations	6	30.0
Incorrect Execution Order	7	35.0
Redundant Operations	5	25.0
Concurrent Operations	2	10.0
Total	20	100.0

Operation Sequence Dependencies. Our analysis identified 20 crash bugs caused by operation dependencies, as summarized in Table 5. The most frequent issue is incorrect execution order, which accounts for 35% of the cases. This issue arises when operations are performed in an improper sequence, such as querying uninitialized resources or deleting active resources, leading to invalid states or crashes. The second most common issue is missing critical operations, observed in 30% of the cases. These bugs occur when essential steps, such as resource initialization or cleanup, are skipped, causing the system to transition into an incomplete or inconsistent state. Redundant operations, responsible for 25% of the crashes, result from repeated execution of the same operation, often leading to resource conflicts, system instability, or excessive resource consumption. Finally, concurrent operations, observed in 10% of the cases, highlight the challenges of managing shared resources in a multi-threaded environment. These issues often manifest as race conditions or deadlocks due to inadequate synchronization.

Insights for VDBFuzz Design. Taken together, these findings directly motivate the design of VDBFuzz. First, the concentration of crashes in storage, indexing, and especially query execution functions suggests that a practical testing tool should prioritize these core APIs when generating test workloads. Second, the prevalence of input-related crashes indicates that systematically mutating user-provided parameters along common failure modes is essential to reveal crash issues. Third, the presence of operation-sequence-dependent crashes highlights the need to explore diverse API invocation orders and state transitions, rather than treating each API call in isolation.

4 WORKFLOW OF VDBFUZZ

The workflow of VDBFuzz is illustrated in Figure 2, which consists of three main stages: 1) *Seed Collection*, 2) *Template-based Input*

Mutation, and 3) *API Sequence Mutation*. These stages are designed to systematically explore crash-inducing behaviors in VDBMSs.

4.1 Preliminary and Definition

To better describe the thoughts behind VDBFuzz, we provide the core definitions of concepts related to API constraints, input mutation, and cross-VDBMS test case transfer.

VDBMS API Constraints. Let $\mathcal{D}$ denote the set of all possible VDBMSs, and let $\mathcal{A}$ represent the set of all API functions exposed by VDBMSs. For a specific VDBMS $D \in \mathcal{D}$, the subset of APIs supported by D is $\mathcal{A}_D \subseteq \mathcal{A}$. Each API function $a \in \mathcal{A}_D$ accepts input parameters $\mathcal{P}_a$ and defines an input space $\mathcal{I}_a$. The valid input space $\mathcal{I}_a^{\text{valid}}$ is determined by a set of constraints $\mathcal{C}_a$, where each constraint $c \in \mathcal{C}_a$ is a predicate:

$$c : \mathcal{I}_a \rightarrow \{\text{true}, \text{false}\}.$$

The valid input space is then:

$$\mathcal{I}_a^{\text{valid}} = \{\mathbf{x} \in \mathcal{I}_a \mid \forall c \in \mathcal{C}_a, c(\mathbf{x}) = \text{true}\}.$$

In addition to exploring the valid input space, VDBFuzz also tests invalid or boundary inputs in $\mathcal{I}_a^{\text{variant}} = \mathcal{I}_a \setminus \mathcal{I}_a^{\text{valid}}$ to identify crash-inducing behaviors.

Input Mutation. An input mutation is a transformation μ applied to an input $\mathbf{x} \in \mathcal{I}_a$, producing a new input $\mathbf{x}' \in \mathcal{I}_a$. Formally:

$$\mu : \mathcal{I}_a \rightarrow \mathcal{I}_a, \quad \mathbf{x}' = \mu(\mathbf{x}).$$

The goal of mutation is to generate diverse inputs, including valid inputs in $\mathcal{I}_a^{\text{valid}}$ and boundary or invalid inputs in $\mathcal{I}_a^{\text{variant}}$.

API Sequence Mutation. API sequence mutation involves modifying the order or structure of a sequence of API calls to uncover crash-inducing behaviors. Let $\mathcal{S} = [s_1, s_2, \dots, s_k]$ represent an ordered sequence of API calls, where each s_i is a tuple $(a, \mathbf{x})$ consisting of an API $a \in \mathcal{A}_D$ and its input $\mathbf{x}$. A sequence mutation ρ generates a new sequence $\mathcal{S}'$ as follows:

$$\rho : \mathcal{S} \rightarrow \mathcal{S}', \quad \mathcal{S}' = \rho(\mathcal{S}).$$

Such mutations include operations such as reordering API calls, omitting critical operations, or introducing redundant calls.

4.2 Seed Collection

The first step in VDBFuzz is the collection of initial seeds, which serve as the foundation for input mutation and sequence mutation.

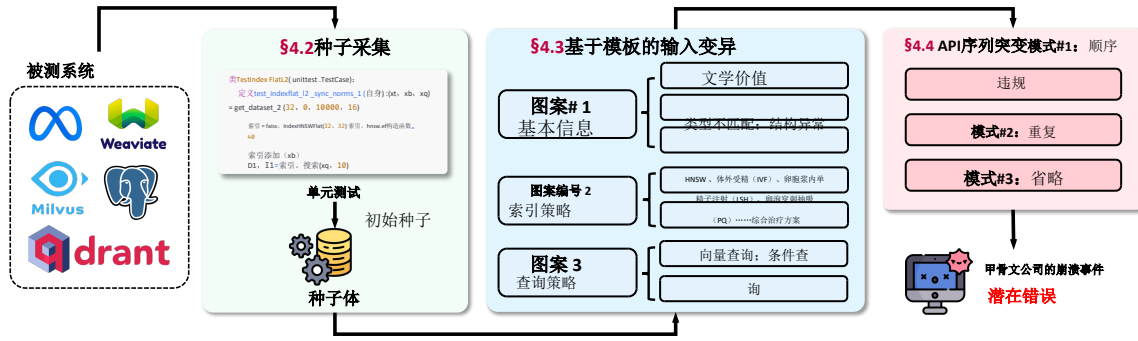


图2: VDBFuzz的整体工作流程。

表5: 由操作依赖性引起的崩溃错误。

依赖问题	频繁性	百分比 (%)
关键操作缺失	6	30.0
错误的执行顺序	7	35.0
冗余操作	5	25.0
并行操作	2	10.0
总计	20	100.0

操作序列依赖性。我们的分析识别出20个由操作依赖性引起的崩溃错误，具体总结见表5。最常见的问题是执行顺序错误，占案例总数的35%；该问题源于操作执行顺序不当（例如查询未初始化资源或删除活跃资源），导致系统状态无效或崩溃。第二常见问题是关键操作缺失，出现于30%的案例中；这类错误发生在跳过资源初始化或清理等必要步骤时，使系统进入不完整或不一致的状态。重复操作（占崩溃案例的25%）源于同一操作的多次执行，常引发资源冲突、系统不稳定或资源消耗过高。最后，并发操作（占10%）凸显了多线程环境中管理共享资源的挑战；这些问题通常表现为因同步不足导致的竞争条件或死锁。

对VDBFuzz设计的启示。综合这些研究结果，我们可明确VDBFuzz的设计方向：首先，系统崩溃现象集中出现在存储、索引处理以及尤其是查询执行功能模块，这表明实际测试工具在生成测试负载时应优先针对这些核心API；其次，输入参数相关崩溃现象普遍存在，说明必须通过系统地改变用户提供的参数并模拟常见故障模式，才能有效发现崩溃问题；第三，依赖操作序列的崩溃现象凸显了需要探索多种API调用顺序和状态转换组合，而非孤立地分析每个API调用。

4 VDBFUZZ 工作流程

VDBFuzz的工作流程如图2所示，包含三个主要阶段：1) 种子收集；2) 基于模板的输入变异；3) API序列变异。这些阶段旨在

系统性地探索VDBMS中的崩溃诱发行为。

4.1 初步定义与界定

为更清晰地阐述VDBFuzz的设计理念，我们提供了与API约束、输入变异及跨VDBMS测试用例迁移相关概念的核心定义。

VDBMS API约束条件。设 D 表示所有可能的VDBMS的集合， A 表示由VDBMS提供的所有API函数的集合。对于特定的VDBMS $D \in D$ ， D 支持的API子集为 AD 。Each API function $a \in AD$ 接受输入参数 P_a 并定义一个输入空间 I_a 。有效的输入空间 I_a^{valid} 由一组约束条件 C_a 确定，其中每个约束条件 $c \in C_a$ 都是一个谓词：

$$c: I_a \rightarrow \{true, false\}.$$

因此，有效的输入空间为：

$$I_a^{有效} = \{x \in I_a \mid c \in C_a, c(x) = true\}.$$

除了探索有效输入空间外，VDBFuzz还测试了 $I_{varianta} = I_a \setminus I_a^{valid}$ 范围内的无效或边界输入，以识别导致系统崩溃的行为。

输入变异。输入变异是指施加于输入 $x \in I_a$ ，producing a new input $x' \in I_a$ 上的变换 μ 。形式化表述如下：

$$\mu: I_a \rightarrow I_a, \quad x' = \mu(x).$$

突变的目标是生成多样化的输入，包括 I_a^{valid} 中的有效输入以及 $I_{varianta}$ 中的边界或无效输入。

API序列变异。API序列变异是指通过修改API调用序列的顺序或结构来揭示导致系统崩溃的行为。设 $S = [s_1, s_2, \dots, s_k]$ 表示一个有序的API调用序列，其中每个 s_i 都是一个元组 (a, x) ，包含API $a \in AD$ 及其输入 x 。序列变异 ρ 可按以下方式生成新序列 S' ：

$$\rho: S \rightarrow S', \quad S' = \rho(S).$$

此类变异包括重新排列API调用顺序、省略关键操作或引入冗余调用等操作。

4.2 种子收集

VDBFuzz的第一步是收集初始种子序列，这些序列作为输入突变和序列突变的基础。

Table 6: Summary of indexing strategies and their mutation rules.

Index	Key Parameters	Constraints	Mutation Examples
HNSW [40]	M, ef_{cons}, ef	$M, ef_{cons}, ef \in \mathbb{Z}^+, ef_{cons} \geq M$	$M < 0, ef_{cons} < M, ef$ set to extreme values
IVF [18]	n_{lists}, n_{probe}	$n_{lists}, n_{probe} \in \mathbb{Z}^+, n_{probe} \leq n_{lists}$	$n_{probe} > n_{lists}, n_{lists} = -1$
PQ [28]	$n_{subvectors}, n_{bits}$	$n_{subvectors}, n_{bits} \in \mathbb{Z}^+, \dim \bmod n_{subvectors} = 0$	$n_{subvectors} = -1, n_{bits} = 0, \dim \bmod n_{subvectors} \neq 0$
LSH [31]	L, k, r	$L, k \in \mathbb{Z}^+, r \in \mathbb{R}^+, r > 0$	$L = 0, r = -1$, extreme values for L, k, r
Flat [18]	metric	metric $\in \{L2, \text{Inner Product}, \text{Cosine}\}$	Mutate metric to unsupported values (e.g., manhattan)
ANNOY [51]	n_{trees}, k	$n_{trees} \in \mathbb{Z}^+, 1 \leq k \leq D $	$n_{trees} = -1, k > D $, or extreme k that degrade performance

These seeds are derived from the Python client libraries of the target VDBMSs, leveraging their built-in unit tests and example scripts [15, 60]. The purpose of this step is to capture representative and valid API usages, ensuring that the seeds reflect real-world interactions with the system under test (SUT). To construct the initial seed corpus, VDBFuzz first analyzes the unit test suites provided by the VDBMS. Each test case is parsed to extract API calls, input parameters, and their dependencies. For example, a unit test for vector indexing might include API calls for creating an index, adding vectors, and performing queries. These calls are recorded in sequence, preserving their logical relationships. Additionally, example scripts and usage documentation are processed in a similar manner to supplement the seed corpus with diverse API patterns not covered by the unit tests. The resulting seed corpus is a structured collection of valid API calls, each represented as a tuple $(a, \mathbf{x})$, where a is the API function and $\mathbf{x}$ is the input parameter set.

4.3 Pattern-based Input Mutation

To systematically uncover boundary inputs to trigger bugs in VDBMSs, we formalize input generation and mutation strategies into reusable patterns. These patterns are specifically designed to construct boundary inputs that expose vulnerabilities in the handling of edge cases, invalid configurations, and unexpected parameter interactions. Below, we describe the patterns for general data mutation, index strategy mutation, and query strategy mutation.

Pattern 1: General Data Mutation. General data mutation systematically generates inputs in $\mathcal{I}_a^{\text{variant}}$, targeting common failure modes such as type errors, boundary violations, and structural inconsistencies. These mutations aim to test the robustness of the database's parsers, execution engines, and indexing mechanisms when handling edge cases. Based on our analysis, approximately 12.3% of observed bugs result from extreme numerical values, 38.6% from zero or null values, and 14.0% from malformed or crafted data structures. To address these failure modes, we define the following rules for generating boundary literal values, mismatched types, malformed structures, and mismatched dimensions.

Sub-pattern 1.1: Boundary Literal Value. This sub-pattern focuses on constructing boundary literal values. These include numerical limits such as minimum, maximum, and zero, as well as structural emptiness, such as NULL, empty strings, empty lists, and empty dictionaries. For example, numerical boundaries like -99999 (minimum) and 99999 (maximum) can be used to test the behavior under extreme conditions. Similarly, structural emptiness, such as an empty list (`[]`) or an empty dictionary (`{}`), helps evaluate how the system handles uninitialized or missing data. Formally,

boundary values can be defined as:

$$\mathcal{I}_a^{\text{boundary}} \rightarrow \{\pm 0.999...99, \pm 999...99, 0, \text{NULL}, '', [], \{\}\}$$

Sub-pattern 1.2: Mismatched Types. This sub-pattern targets inputs in $\mathcal{I}_a^{\text{variant}}$ where the type of a value does not match the expected type. Examples include providing a string where an integer is expected, or passing a list (`[1, 2, 3]`) where a floating-point value is required. Such mutations can expose flaws in type-checking mechanisms or unexpected type coercion behaviors. Formally, invalid types are defined as:

$$\mathcal{I}_a^{\text{type}} = \{\mathbf{x} \in \mathcal{I}_a \mid \mathbf{x} \notin \text{ExpectedType}\}.$$

Sub-pattern 1.3: Malformed Structure. This sub-pattern generates inputs with invalid structures to test the resilience of parsers and execution engines. Such inputs include corrupted JSON objects, incomplete filter definitions, or misaligned data structures. For example, a malformed JSON object like `{"key": }` (missing value) or an incomplete filter like `filter = {"field": }` (missing condition) can reveal vulnerabilities in input validation. Formally, malformed structures are defined as:

$$\mathcal{I}_a^{\text{malformed}} = \{\mathbf{x} \in \mathcal{I}_a \mid M_{\text{json}}(\mathbf{x}) \vee M_{\text{filter}}(\mathbf{x}) \vee M_{\text{schema}}(\mathbf{x})\},$$

Pattern 2: Indexing Strategy Mutation. Indexing strategy mutation focuses on systematically exploring invalid or boundary configurations of key parameters used for creating vector indexes. Incorrect or extreme configurations, such as those in HNSW, IVF, PQ, and LSH indexing methods, can lead to indexing failures, degraded query performance, inaccurate results, or even system crashes. This pattern evaluates how well the VDBMS handles misconfigurations, extreme values, and invalid combinations. Based on the abstraction of indexing methods, we define two overarching mutation patterns.

Sub-pattern 2.1: Single Indexing Strategy Mutation. This pattern focuses on parameters of individual indexing methods (e.g., HNSW, IVF, PQ, and LSH). Each method defines a unique set of key parameters, such as M, ef_{cons} , and ef in HNSW, or n_{lists} and n_{probe} in IVF. These parameters have specific constraints (e.g., positivity, ranges, interdependencies), violations of which can lead to indexing errors or performance degradation. The general mutation space for single indexing strategies is defined as:

$$\mathcal{I}_{\text{index}}^{\text{variant}} = \{p_i \mid p_i \notin \text{ValidRange}(p_i) \vee \text{Violation}(p_1, p_2, \dots, p_n)\},$$

where p_i represents a parameter, and constraints are method-specific (e.g., $ef_{cons} \geq M$ in HNSW, $n_{probe} \leq n_{lists}$ in IVF). Table 6 summarizes the parameters, constraints, and mutation strategies for the most commonly used indexing methods.

表6: 索引策略及其变异规则总结。

索引	关键参数	约束条件	突变示例
HNSW [40]	M, ef_{cons}, ef	$M, ef_{cons}, ef \in \mathbb{Z}^+, ef_{cons} \geq M$	$M < 0, ef_{cons} < M, ef$ 设为极端值
IVF [18]	n 列表, n 探测器	n 列表, n 探测器 $\mathbb{Z}^+, n_{probe} \leq n_{lists}$	$n_{probe} > n_{lists}, n_{lists} = -1$
PQ [28]	n 个子向量, n 位	n 个子向量, n 位 $\mathbb{Z}^+$, 维度模 $n_{subvectors}=0$	$n_{subvectors} = -1, n_{bits} = 0$, 维度模 $n_{subvectors} = 0$
LSH [31]	L, k, r	$L, k \in \mathbb{Z}^+, r \in \mathbb{R}^+, r > 0$	$L = 0, r = -1, L, k, r$ 的极值
平坦 [18]	指标	度量函数属于集合 {L2 范数、内积、余弦函数}。	将度量值更改为不支持的值 (例如: manhattan)
烦人 [51]	n_{trees}, k	$n_{trees} \in \mathbb{Z}^+, 1 \leq k \leq D $	$n_{trees} = -1, k > D $, 或导致性能下降的极端 k 值

这些测试用例源自目标VDBMS系统的Python客户端库，充分利用了其内置的单元测试和示例脚本[15, 60]。此步骤旨在捕获具有代表性和有效性的API使用场景，确保测试用例能够反映与被测系统（SUT）的真实交互情况。为构建初始测试集，VDBFuzz首先分析 VDBMS 提供的单元测试套件：对每个测试用例进行解析以提取API调用、输入参数及其依赖关系（例如，向量索引相关的单元测试可能包含创建索引、添加向量及执行查询等API调用），并按逻辑顺序记录这些调用关系；同时采用类似方式处理示例脚本和使用文档，以补充单元测试未涵盖的多样化API模式。最终生成的测试集是一个结构化的有效API调用集合，每个调用均表示为元组 $(a, \mathbf{x})$ ，其中 a 为API函数， $\mathbf{x}$ 为输入参数集。

4.3 基于模式的输入突变

为系统性地识别可能导致VDBMS系统漏洞的边界输入，我们将输入生成与变异策略规范化为可复用模式。这些模式专门用于构建能够暴露边缘场景处理缺陷、无效配置问题以及意外参数交互风险的边界输入。下文将分别阐述通用数据变异模式、索引策略变异模式及查询策略变异模式的具体实现方案。

模式1: 通用数据变异。通用数据变异方法可系统性地为 Ivarianta 生成测试输入，针对类型错误、边界违规及结构不一致等常见故障模式。这些变异旨在检验数据库解析器、执行引擎及索引机制在处理边缘场景时的稳健性。根据我们的分析，约12.3%的观测到的错误源于极端数值，38.6%源于零值或空值，14.0%源于格式错误或人为构造的数据结构。为应对这些故障模式，我们制定了以下规则用于生成边界字面值、类型不匹配、格式错误结构以及维度不匹配的情况。

*子模式 1.1: 边界字面值。*该子模式专注于构建边界字面值，包括最小值、最大值及零等数值边界，以及 NULL、空字符串、空列表和空字典等结构空值。例如，使用 -99999（最小值）和 99999（最大值）等数值边界可测试系统在极端条件下的行为；类似地，空列表 $([])$ 或空字典 $(\{\})$ 等结构空值有助于评估系统如何处理未初始化或缺失的数据。严格来

说，边界值可定义为：

$$I_a^{\text{边界}} = \{\pm 0.999...99, \pm 999...99, 0, \text{NULL}, '', [], \{\}\}$$

*子模式 1.2: 类型不匹配。*该子模式针对 Ivarianta 中输入值的类型与预期类型不符的情况。例如：在预期为整数的位置传入字符串，或在需要浮点型值的位置传递列表 $([1, 2, 3])$ 。此类错误可能导致类型检查机制存在缺陷或引发意外的类型强制转换行为。形式上，无效类型定义如下：

$$I_{\text{type}} = \{\mathbf{x} \mid \mathbf{a} \mid \mathbf{x} \text{ ExpectedType}\}.$$

*子模式 1.3: 无效结构。*该子模式生成具有无效结构的输入数据，用于测试解析器和执行引擎的容错能力。此类输入包括损坏的 JSON 对象、不完整的过滤条件或数据结构排列错误。例如，格式错误的 JSON 对象（如 $\{\text{"key"} : \}$ 缺值）或不完整的过滤条件（如 $\text{filter} = \{\text{"field"} : \}$ 缺条件）均可能暴露输入验证中的漏洞。严格而言，无效结构定义如下：

$$I_a^{\text{形态异常}} = \{\mathbf{x} \mid \mathbf{a} \mid M_{\text{json}}(\mathbf{x}) \quad M_{\text{filter}}(\mathbf{x}) \quad M_{\text{schema}}(\mathbf{x})\}, \text{模}$$

式2: 索引策略变更。索引策略变更旨在系统性地探究用于创建向量索引的关键参数所存在的无效或边界配置。诸如 HNSW、IVF、PQ 及 LSH 索引方法中存在的错误或极端配置可能导致索引失败、查询性能下降、结果不准确甚至系统崩溃。该模式用于评估 VDBMS 处理配置错误、极端值及无效组合的能力。基于对索引方法的抽象化分析，我们定义了两种核心变更模式。

*子模式 2.1: 单一索引策略变异。*该模式重点研究各类独立索引方法（如 HNSW、IVF、PQ 和 LSH）的参数设置。每种方法都定义了一组独特的关键参数——例如 HNSW 中的 M 、 ef_{cons} 和 ef ，或 IVF 中的 n_{lists} 和 n_{probe} 。这些参数具有特定约束条件（如非负性、取值范围、相互依赖关系），违反这些约束可能导致索引错误或性能下降。单一索引策略的通用变异空间定义如下：

$$I_{\text{index}}^{\text{variant}} = \{p_i \mid p_i \text{ ValidRange}(p_i) \text{ 违反条件 } (p_1, p_2, \dots, p_n)\}, \text{其中 } p_i \text{ 代表参数, 约束条件因方法而异 (例如 IVF 中的 } ef_{cons} \geq M \text{ in HNSW, } n_{probe} \leq n_{lists}\text{)。表6总结了最常用索引方法的参数、约束条件及变异策略。}$$

Table 7: Constraints and mutations for vector queries.

Parameter	Constraints and Mutations
k (number of results)	Valid: $k \in \mathbb{Z}^+, 1 \leq k \leq D $ Mutations: • $k = 0$ (empty results) • $k < 0$ (e.g., $k = -1$) • $k > D$ (exceeds dataset size) • Non-integer types (e.g., $k = 3.5$, $k = \text{"ten"}$)
$\vec{q}$ (query vector)	Valid: $\vec{q} \in \mathbb{R}^d, d > 0$ Mutations: • Incorrect dimensionality ($d' \neq d$) • Malformed values (e.g., NaN, Inf) • Extreme values (e.g., excessively large)
M (distance metric)	Valid: $M \in \mathcal{M}_{\text{valid}}$ Mutations: • Unsupported metrics (e.g., cosine for L2)
o (offset for pagination)	Valid: $o \in \mathbb{Z}, o \geq 0$ Mutations: • Negative values (e.g., $o = -1$) • Non-integer values (e.g., $o = 1.5$) • Excessively large offsets (e.g., $o > D$)

Pattern 2.2: Composite Indexing Strategy Mutation. This pattern targets combinations of multiple indexing methods, such as IVF-PQ or HNSW-PQ, where parameters from different methods must satisfy simultaneous constraints. The mutation space for composite strategies is defined as:

$$\mathcal{I}_{\text{composite}}^{\text{variant}} = \bigcup_{i=1}^n \mathcal{I}_i^{\text{variant}},$$

where $\mathcal{I}_i^{\text{variant}}$ represents the invalid parameter space of the i -th component indexing method. For example, in IVF-PQ, $n_{\text{subvectors}}$ can be set to a non-divisible value while $n_{\text{probe}} > n_{\text{lists}}$, exposing flaws in parameter validation across strategies.

Pattern 3: Query Parameter Mutation. Query parameter mutation focuses on exploring the robustness of VDBMS query functionalities by systematically testing various query parameters. Querying is the core operation in vector databases, and the behavior of queries is shaped by parameters such as the number of results to return, query vectors, filtering conditions, distance metrics, and pagination settings. Misconfigured parameters, overly complex filters, or invalid combinations can lead to logic errors, degraded performance, or even system failures. This pattern is divided into two sub-patterns: *vector queries* and *predicate queries*.

Sub-pattern 3.1: Vector Queries. Vector queries are defined by parameters such as k (the number of results to return), the query vector $\vec{q}$, the distance metric M , and the pagination offset o . These parameters must satisfy specific constraints, as summarized in Table 7. Mutation strategies explore invalid configurations, such as setting k to values outside its valid range, introducing malformed query vectors, using unsupported distance metrics, or applying invalid pagination offsets. Formally, the invalid parameter space for vector queries is defined as: Formally, the invalid parameter space

for vector queries can be defined as:

$$\mathcal{I}_{\text{vector}}^{\text{variant}} = \{(k, \vec{q}, M, o) \mid \begin{cases} k \notin \mathbb{Z}^+ \text{ or } k > |D|, \\ \vec{q} \notin \mathbb{R}^d, \\ M \notin \mathcal{M}_{\text{valid}}, \\ o \notin \mathbb{Z} \text{ or } o < 0 \end{cases}\}.$$

Sub-pattern 3.2: Predicate Queries. Predicate queries extend vector queries by introducing filtering conditions F , which constrain results based on metadata attributes or logical expressions. These filters enable more complex query scenarios, such as combining similarity search with metadata-based constraints.

Filters F can fail due to syntax errors (e.g., mismatched parentheses, invalid operators) or semantic errors (e.g., referencing nonexistent fields, unsupported operations, or logically contradictory conditions). For example, a filter might reference a non-existent metadata field, apply a range query (gte/lte) on a string field, or include logically contradictory constraints such as `field > 10 AND field < 5`. Geospatial filters may also fail due to malformed coordinates or nonsensical ranges. Additionally, overly complex filters with deeply nested AND/OR combinations can degrade query performance, especially when combined with similarity search. Formally, the invalid space for predicate queries is defined as:

$$\mathcal{I}_{\text{predicate}}^{\text{variant}} = \{(k, \vec{q}, M, o, F) \mid \begin{cases} (k, \vec{q}, M, o) \in \mathcal{I}_{\text{vector}}^{\text{variant}}, \\ F \in \mathcal{F}_{\text{invalid}} \end{cases}\}.$$

Here, $\mathcal{F}_{\text{invalid}}$ represents the space of invalid filters, encompassing syntax errors, semantic errors, and logical contradictions.

4.4 API Sequence Mutation.

API sequence mutation systematically evaluates the robustness and correctness of VDBMSs by mutating sequences of API calls. Let $\Sigma = \{s_1, s_2, \dots, s_n\}$ denote the set of all valid API operations, where each s_i represents an individual API call. A valid API sequence is defined as an ordered list of calls:

$$S_{\text{valid}} = \langle s_1, s_2, \dots, s_k \rangle,$$

where each s_i must satisfy its precondition $P(s_i)$, and the sequence collectively ensures valid state transitions. For example, a normal API sequence might comprise operations such as `connect`, `create_collection`, `add_object`, `search`, and `close`, executed in the correct order. Deviations from this order or sequence can lead to various invalid states. Mutation strategies systematically violate one or more constraints of S_{valid} . These strategies can be categorized into three primary types: *Order Violation*, *Repetition*, and *Omission*, as summarized in Table 8.

Pattern 1: Order Violation. Order violation involves reordering API operations in a way that disrupts the sequence's logical flow and violates the precondition constraints of one or more operations. For instance, calling `search` before `add_object` would result in a state where the search operation is executed on an empty or uninitialized state, thereby violating the expected preconditions.

Pattern 2: Repetition. Repetition of operations focuses on testing robustness when specific API calls are invoked multiple times, potentially exceeding their intended usage. For example, repeatedly invoking `connect` without corresponding `close` calls can lead to resource exhaustion, such as the depletion of available connections.

表7: 向量查询的约束条件与变异类型。

参数	约束条件与突变
k (结果数量)	有效格式: $k \in \mathbb{Z}^+, 1 \leq k \leq D$ 突变: $k=0$ (空结果) $k < 0$ (e.g., $k = -1$) $k > D$ (超过数据集大小) - 非整型类型 (例如: $k=3.5$, $k=“十”$)
q (查询向量)	有效: $q \in \mathbb{R}^d, d>0$ 突变: - 维度不正确 ($d=d$) - 无效值 (例如: NaN, Inf) - 极端值 (例如: 过大)
M (距离度量)	有效: $M \in \mathcal{M}$ 有效 突变: 不支持的度量指标 (例如: L2范数下的余弦值)
o (分页偏移量)	有效: $o \in \mathbb{Z}, o \geq 0$ 突变: - 负值 (例如: $o=-1$) - 非整数值 (例如, $o=1.5$) - 偏移量过大 (例如: $o > D$)

模式2.2: 复合索引策略变异。该模式针对多种索引方法 (如 IVFPQ 或 HNSW-PQ) 的组合方案, 要求不同方法的参数同时满足约束条件。复合策略的变异空间定义如下:

$$\mathcal{I}_{\text{composite}}^{\text{variant}} = \bigcup_{i=1}^n \mathcal{I}_i^{\text{variant}},$$

其中 $\mathcal{I}_i^{\text{variant}}$ 表示第 i 个参数空间的无效值范围/
组件索引方法。例如, 在 IVF-PQ 中, $n_{\text{subvectors}}$ 可设置为不可整除的数值, 而 n_{probe} 大于 列表数量 n_{lists} , 这暴露出不同策略间参数验证存在的缺陷。

模式3: 查询参数变更。查询参数变更旨在通过系统地测试不同查询参数, 来评估 VDBMS 查询功能的稳健性。查询是向量数据库的核心操作, 其运行效果受返回结果数量、查询向量、过滤条件、距离度量及分页设置等参数影响。配置错误的参数、过于复杂的过滤条件或无效组合可能导致逻辑错误、性能下降甚至系统故障。该模式可进一步细分为两个子模式: **向量查询与谓词查询**。

子模式 3.1: 向量查询。向量查询由以下参数定义: k (返回的结果数量)、查询向量 q 、距离度量 M 以及分页偏移量 o 。这些参数必须满足特定约束条件, 具体如表7所示。变异策略会探索无效配置, 例如将 k 设置为超出有效范围的值、引入格式错误的查询向量、使用不受支持的距离度量或应用无效的分页偏移量。形式上, 向量查询的无效参数空间定义如下:

for vector queries can be defined as:

$$\mathcal{I}_{\text{vector}}^{\text{variant}} = \{(k, \vec{q}, M, o) \mid \begin{cases} k \notin \mathbb{Z}^+ \text{ or } k > |D|, \\ \vec{q} \notin \mathbb{R}^d, \\ M \notin \mathcal{M}_{\text{valid}}, \\ o \notin \mathbb{Z} \text{ or } o < 0 \end{cases}\}.$$

子模式 3.2: 谓词查询。谓词查询通过引入过滤条件 F 来扩展向量查询, 这些条件可根据元数据属性或逻辑表达式对结果进行约束。此类过滤器可支持更复杂的查询场景, 例如将相似性搜索与基于元数据的约束条件相结合。

过滤器 F 可能因语法错误 (如括号不匹配、运算符无效) 或语义错误 (如引用不存在的字段、使用不受支持的操作或存在逻辑矛盾的条件) 而失败。例如, 过滤器可能引用不存在的元数据字段、对字符串字段应用范围查询 (`gte/lte`), 或包含逻辑矛盾的约束条件 (如 “字段值大于10且小于5”)。地理空间过滤器也可能因坐标格式错误或范围定义不合理而失败。此外, 包含大量嵌套 AND/OR 组合的复杂过滤器会降低查询性能, 尤其是在与相似性搜索结合使用时。严格来说, 谓词查询中无效的空间定义如下:

$$\mathcal{I}_{\text{predicate}}^{\text{variant}} = \{(k, \vec{q}, M, o, F) \mid \begin{cases} (k, \vec{q}, M, o) \in \mathcal{I}_{\text{vector}}^{\text{variant}}, \\ F \in \mathcal{F}_{\text{invalid}} \end{cases}\}.$$

此处, $\mathcal{F}_{\text{invalid}}$ 代表无效过滤器的空间, 涵盖语法错误、语义错误及逻辑矛盾。

4.4 API序列突变。

API序列突变法通过变异API调用序列来系统地评估VDBMS的稳健性和正确性。设 $\vec{s} = \{s_1, s_2, \dots, s_n\}$ 表示所有有效API操作的集合, 其中每个 s_i 代表一个独立的API调用。有效的API序列被定义为有序调用列表:

$$S_{\text{valid}} = \langle s_1, s_2, \dots, s_k \rangle,$$

其中每个 s_i 都必须满足其前提条件 $P(s_i)$, 且整个序列共同确保状态转换的有效性。例如, 一个标准API序列可能包含 `connect`、`create_collection`、`add_object`、`search` 和 `close` 等操作, 并按正确顺序执行。偏离此顺序或序列可能导致各种无效状态。变异策略会系统地违反 S_{valid} 的一个或多个约束条件。这些策略可分为三种主要类型: **顺序违规**、**重复操作** 和 **遗漏操作**, 如表8所示。

模式1: 顺序违规。顺序违规指以破坏操作逻辑流程、违反一个或多个操作前置条件的方式重新排列API操作顺序。例如, 在调用 `add_object` 之前执行 `search` 操作, 会导致搜索操作在空状态或未初始化状态下执行, 从而违反预期的前置条件。

模式2: 重复执行。操作的重复执行旨在测试当特定API调用被多次调用时 (可能超出其预期使用场景) 的稳健性。例如, 若在未执行相应关闭操作的情况下反复调用 `connect` 函数, 可能导致资源耗尽 (如可用连接数量耗尽)。

Table 8: API sequence mutation strategies.

Mutation Strategy	Mutation Description
Order Violation	Reorder operations in the sequence S such that at least one s_i violates its precondition $P(s_i)$. Example: search before add_object
Repetition of Operations	Repeat an operation $s_r \in S$. Example: connect $\rightarrow$ connect
Omission of Operations	Remove one or more critical operations from S . Example: Skipping add_object

Pattern 3: Omission. Omission of operations involves skipping critical API calls required to maintain valid state transitions. For example, omitting the add_object call that expects objects to be added to a collection before a search operation can leave the system in an incomplete state, leading to failed expectations or errors.

The three mutation strategies collectively aim to create scenarios that stress the system’s ability to maintain correctness under adverse conditions. Additionally, these strategies are invaluable for testing concurrency and distributed VDBMSs, where interactions between multiple API calls can lead to race conditions, deadlocks, or inconsistencies in shared resources.

5 EVALUATION

5.1 Implementation

The implementation of VDBFuzz is designed to uncover boundary and edge-case issues across different types of VDBMSs, including native VDBMSs (e.g., Weaviate [61], Milvus [35], and Qdrant [46]), VDBMS libraries (e.g., Faiss [16], HNSWLib [40], and Annoy [51]), and extended VDBMSs (e.g., pgvector [42] and sqlite-vec [3]). Native VDBMSs typically expose their core functionalities via HTTP or gRPC APIs, while VDBMS libraries and extended VDBMSs rely on Python-based client interfaces for interaction. To ensure comprehensive testing across these diverse implementations, we tailored VDBFuzz to the specific characteristics of each category.

For native VDBMS like Weaviate, Milvus, and Qdrant, we leverage the Python client libraries to construct a fuzz harness for generating and executing API sequences, while relying on hooks at the HTTP and gRPC communication to perform seed mutation. By intercepting the communication between the Python client and the database server, we apply mutation strategies directly to the requests being sent to the core implementation. For VDBMS libraries such as Faiss, HNSWLib, and Annoy, which do not expose API endpoints, and for extended VDBMS systems like PostgreSQL, we implemented our fuzzing strategies directly at the Python layer. This involved creating mutation strategies to test various input parameters and configurations during indexing and querying.

Specifically, we have implemented a prototype of VDBFuzz using over 4K lines of code (LoC) with Python, excluding any third-party libraries or open-source tools. The HTTP and gRPC interceptors, comprising around 1,300 lines of code, are responsible for capturing and mutating communication payloads. The core fuzzer logic,

including modules for API sequence mutation, data mutation, and template generation, takes up approximately 3,300 lines and is designed to generate diverse and complex test cases. The template generation module ensures that inputs adhere to the structural requirements of the target APIs while systematically introducing invalid or boundary values. Additionally, auxiliary modules handle tasks such as parsing test outputs, managing mutation strategies, and integrating with the fuzz harness.

5.2 Evaluation Setup

Running Environment. The evaluation of VDBFuzz was conducted on a high-performance server running Ubuntu Linux 22.04, equipped with two AMD EPYC Milan 7713 CPUs (2.0 GHz, 64 cores, 128 threads each), 512 GB of RAM, and four 7.68 TB NVMe SSDs. To ensure comprehensive coverage analysis across the diverse implementations of VDBMSs written in C++, Rust, and Go, we employed language-specific workflows. For Rust-based systems (Qdrant), the code was compiled with coverage instrumentation enabled (`-Cinstrument-coverage`), and profiling data was collected as `.profraw` files, which were merged using `llvm-profdata` and analyzed with `grcov` [37] to generate branch-level HTML reports. For Go-based implementations (Weaviate and Milvus), we utilized `goc` [47], a coverage tool specifically designed for the Go language, to instrument the code and generate detailed coverage reports during test execution.

Research Questions. To evaluate the performance of VDBFuzz, we aim to answer the following research questions (RQs):

- **RQ1: Comparison with Existing Tools.** How does VDBFuzz compare to state-of-the-art fuzzers and testing tools in terms of code coverage across different VDBMSs?
- **RQ2: Ablation Study.** How does each mutation strategy individually contribute to the overall performance of VDBFuzz?
- **RQ3: Real-World Bugs Discovered.** What types of real-world boundary and edge-case vulnerabilities were discovered by VDBFuzz in tested VDBMSs?

5.3 Comparison with Existing Tools (RQ1)

To evaluate the effectiveness of VDBFuzz compared to existing state-of-the-art API fuzzing tools, we conducted a series of experiments using the three native VDBMSs in our evaluation: Weaviate, Qdrant, and Milvus. The tools selected for comparison were RESTler [13] and Schemathesis [14], as both are widely adopted for OpenAPI testing and have demonstrated strong results in identifying API vulnerabilities and achieving high code coverage.

Baseline Setup. For each VDBMS, we used its publicly available OpenAPI schema to guide the fuzzing process. Each tool was run for two hours (120 minutes) on each VDBMS to ensure fair resource allocation and allow sufficient time to explore the API space.

- **RESTler:** RESTler was configured in its default fuzzing mode, which performs stateful API fuzzing by generating API sequences based on dependencies inferred from the OpenAPI schema. It prioritizes breadth-first exploration of the API space.
- **Schemathesis:** Schemathesis was executed in its default property-based testing mode, which generates test cases by mutating inputs to API endpoints based on the OpenAPI schema. It focuses on identifying schema violations and edge cases.

表8: API序列突变策略。

突变策略	突变描述
订单违规	对序列S中的操作进行重新排序, 使得
	至少有一个 s_i 违反其前置条件 $P(s_i)$ 。 示例: 添加对象前进行搜索
操作的重复执行	重复执行操作 $s \in S$ 。示例: connect connect
省略操作步骤	从S中移除一项或多项关键操作。 示例: 跳过 add_object

模式3: 省略。运算省略指跳过维持有效状态转换所需的键API调用。例如, 在搜索操作之前省略需要向集合添加对象的add_object调用, 可能导致系统处于未完成状态, 进而引发预期失败或错误。

这三种mutational策略共同旨在构建能够检验系统在不利条件下保持正确性的场景。此外, 这些策略对于测试并发性及分布式VDBMS系统尤为宝贵——在这些系统中, 多个API调用之间的交互可能导致竞争条件、死锁或共享资源中的不一致性问题。

5 评估

5.1 实施; 执行

VDBFuzz的实现旨在检测各类VDBMS中的边界问题与边缘情况问题, 涵盖原生VDBMS (如Weaviate[61]、Milvus[35]和Qdrant[46])、VDBMS 库 (如Faiss[16]、HNSWLib[40]和Annoy[51]) 以及扩展型VDBMS (如pgvector[42]和sqlite-vec[3])。原生VDBMS通常通过HTTP或gRPC API提供核心功能, 而VDBMS 库及扩展型VDBMS则依赖基于Python的客户端接口进行交互。为确保对这些不同实现进行全面测试, 我们根据各类型系统的具体特性对VDBFuzz进行了定制化设计。

对于Weaviate、Milvus和Qdrant等原生VDBMS 库, 我们利用Python客户端库构建模糊测试框架来生成并执行API调用序列, 并通过HTTP和gRPC通信接口实现种子参数变异。通过拦截Python客户端与数据库服务器之间的通信数据, 我们将变异策略直接应用于发送至核心系统的请求。而对于未提供API接口的VDBMS 库 (如Faiss、HNSWLib 和 Annoy), 以及PostgreSQL等扩展型VDBMS 系统, 则直接在Python层面实现模糊测试策略——具体包括设计用于测试索引创建和查询过程中各类输入参数及配置方案的变异策略。

具体而言, 我们使用Python语言开发了VDBFuzz的原型系统, 代码量超过4000行 (不含第三方库或开源工具)。其中包含约1300行代码的HTTP和gRPC拦截器负责捕获并篡改通信载荷; 核心模糊测试逻辑模块 (涵盖API序列变异、数据变异

及模板生成功能) 约占3300行代码, 旨在生成多样化且复杂的测试用例。模板生成模块确保输入参数符合目标API的结构要求, 并系统性地引入无效值或边界值; 辅助模块则负责解析测试输出、管理变异策略以及与模糊测试框架集成等任务。

5.2 评估设置

运行环境。VDBFuzz的评估测试在一台运行Ubuntu Linux 22.04系统的高性能服务器上完成, 该服务器配备两颗AMD EPYC Milan 7713处理器 (主频2.0 GHz, 各含64个核心和128个线程)、512 GB内存以及四块7.68 TB NVMe固态硬盘。为确保对用C++、Rust和Go语言实现各类VDBMS实现方案进行全面覆盖分析, 我们采用了针对不同编程语言定制的工作流程: 对于基于Rust的系统 (Qdrant), 代码编译时启用了覆盖率监控功能 (-Cinstrument-coverage), 生成的性能分析数据以profraw 文件形式存储, 随后通过llvm-profdata工具进行合并, 并利用grcov[37]工具生成分支级别的HTML报告; 而对于基于Go的语言实现 (Weaviate和Milvus), 则使用专为Go语言设计的覆盖率工具goc[47]对代码进行监控, 并在测试执行过程中生成详细的覆盖率报告。

研究问题。为评估VDBFuzz的性能, 我们旨在回答以下研究问题 (RQs):

- **RQ1: 与现有工具的比较。**在不同VDBMS中, VDBFuzz在代码覆盖率方面与最先进的模糊测试工具及测试工具相比如何?
- **RQ2: 消融研究。**各种突变策略如何分别影响VDBFuzz的整体性能?
- **RQ3: 已发现的实际漏洞。**VDBFuzz在测试的VDBMS系统中发现了哪些类型的现实边界漏洞及边缘场景漏洞?

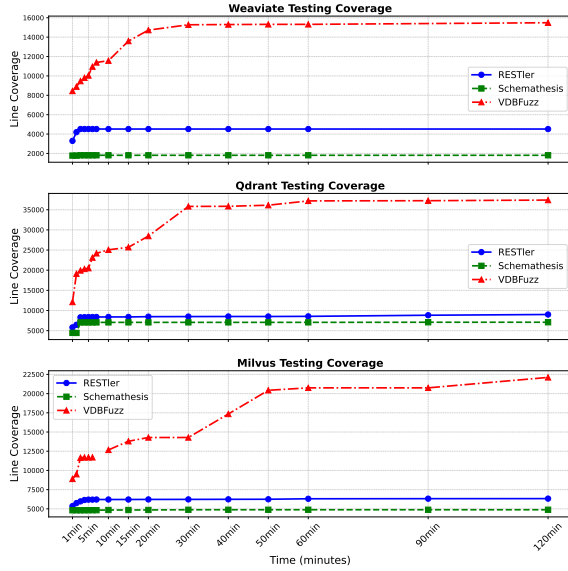
5.3 与现有工具的比较 (RQ1)

为评估VDBFuzz相较于现有先进API模糊测试工具的有效性, 我们使用评估所用的三种原生VDB管理系统 (Weaviate、Qdrant和Milvus) 开展了一系列实验。选择用于对比的工具是RESTler[13]和Schemathesis[14], 这两种工具均广泛应用于OpenAPI测试, 并在识别API漏洞及实现高代码覆盖率方面表现出色。**基线设置:** 针对每个VDBMS, 我们均采用其公开可用的OpenAPI模式来指导模糊测试流程; 每种工具在每个VDBMS上运行两小时 (120分钟), 以确保资源分配均衡并为探索整个API空间提供充足时间。

- **RESTler:** RESTler被配置为默认模糊测试模式, 该模式通过根据从OpenAPI模式推断出的依赖关系生成API序列来执行有状态API模糊测试, 并优先对API空间进行广度优先探索。
- **架构推导:** 架构推导采用默认的基于属性的测试模式执行, 该模式根据OpenAPI 模式通过修改API 端点的输入参数来生成测试用例, 重点在于识别模式违规情况及边缘用例。

Table 9: VDBFuzz’s improvement in coverage compared to baseline fuzzers across Weaviate, Qdrant, and Milvus.

VDBMS	Baseline	Cov.	VDBFuzz
Weaviate	RESTler	4,519 (7.0%)	+243.0%
	Schemathesis	1,811 (2.8%)	+755.1%
Qdrant	RESTler	8,402 (9.1%)	+345.3%
	Schemathesis	7,050 (7.6%)	+430.6%
Milvus	RESTler	6,331 (11.0%)	+249.2%
	Schemathesis	4,849 (8.4%)	+355.8%

**Figure 3: Code coverage comparison of RESTler, Schemathesis, and VDBFuzz on Weaviate, Qdrant, and Milvus.**

- **VDBFuzz:** VDBFuzz was run in its full mutation mode, which combines API sequence mutation, input data mutation, and template-based indexing and query mutation strategies.

To measure the effectiveness of each tool, we recorded the total line coverage achieved over time. We report the mean line coverage over five runs of VDBFuzz and these baselines.

Comparison Results. Table 9 highlights the overall testing capabilities of VDBFuzz compared to baseline fuzzing tools RESTler and Schemathesis across Weaviate, Qdrant, and Milvus. The results show that VDBFuzz consistently outperformed the baseline tools by a substantial margin in both testing efficiency and effectiveness. On average, VDBFuzz achieved more than 3x the total line coverage of RESTler and over 4x the coverage of Schemathesis.

Specifically, we visualized the testing progress over time in Figure 3 to provide a detailed comparison of line coverage growth across the three VDBMSs. Within the first 10 minutes, VDBFuzz achieved significantly higher line coverage compared to the baseline tools, demonstrating its superior efficiency in exploring API spaces. For instance, on Weaviate, VDBFuzz reached 11,571 lines of code

Table 10: Ablation study on mutation strategies across Weaviate, Qdrant, and Milvus.

Strategy	Weaviate	Qdrant	Milvus
VDBFuzz	15,485	37,403	22,109
VDBFuzz-seed-only	14,842 -643	36,342 -1,061	20,140 -1,969
VDBFuzz-w/o index	15,469 -16	36,493 -910	20,355 -1,754
VDBFuzz-w/o query	15,331 -154	36,550 -853	21,709 -400

coverage within 10 minutes, compared to 4,519 lines for RESTler and 1,811 lines for Schemathesis. Over the entire 120-minute testing period, VDBFuzz maintained its advantage, achieving total coverage of 15,485 lines on Weaviate, 37,403 lines on Qdrant, and 22,109 lines on Milvus. These results represent approximately 3.4x to 4.2x higher coverage than RESTler and 4.5x to 8.5x higher coverage than Schemathesis across the three VDBMSs. Notably, RESTler and Schemathesis exhibited limited growth in coverage after the early stages of testing, with RESTler plateauing after just a few minutes and Schemathesis reaching its peak coverage at a significantly lower level, highlighting their limitations when applied to VDBMS fuzzing. In contrast, VDBFuzz demonstrated consistent improvement throughout the testing period, uncovering increasingly complex API interactions and implementation logic.

5.4 Ablation Study (RQ2)

To evaluate the contribution of different mutation strategies in VDBFuzz, we conducted an ablation study by systematically disabling specific components and measuring their impact on line coverage across Weaviate, Qdrant, and Milvus. Table 10 summarizes the results, showing the total line coverage achieved by the full version of VDBFuzz compared to its ablated variants. The results indicate that the full version of VDBFuzz achieved the highest line coverage across all three VDBMSs, with 15,485 lines for Weaviate, 37,403 lines for Qdrant, and 22,109 lines for Milvus. Disabling specific components resulted in a consistent reduction in coverage, demonstrating the complementary nature of the mutation strategies. Below, we analyze the impact of each strategy:

- **Seed-only.** In this variant, VDBFuzz relied solely on the initial seeds. This resulted in the largest reduction in coverage, with decreases of 643 lines on Weaviate, 1,061 lines on Qdrant, and 1,969 lines on Milvus. These results highlight the importance of edge case mutations in uncovering deeper implementation logic that seed-only approaches fail to explore.
- **Index-specific Mutation.** When index-specific mutation was disabled, the coverage dropped by 16 lines on Weaviate, 910 lines on Qdrant, and 1,754 lines on Milvus. This strategy had a particularly significant impact on Milvus and Qdrant.
- **Query-specific Mutation.** Disabling the query-specific mutation strategy slightly reduced coverage, with decreases of 154 lines on Weaviate, 853 lines on Qdrant, and 400 lines on Milvus. While the impact of this strategy was smaller compared to index-specific mutation, it played a crucial role in improving the exploration of query-related API behaviors, especially on Weaviate, where the reduction was more pronounced.

表9: VDBFuzz在Weaviate、Qdrant和Milvus平台上相较于基线模糊测试工具的覆盖率提升情况。

VDBMS	基准	Cov.	VDBFuzz	
Weaviate	RESTler	4,519 (7.0%)	15,485 (24.0%)	+243.0%
	框架理论	1,811 (2.8%)		+755.1%
Qdrant	RESTler	8,402 (9.1%)	37,403 (40.3%)	+345.3%
	框架理论	7,050 (7.6%)		+430.6%
米尔沃斯	RESTler	6,331 (11.0%)	22,109 (38.5%)	+249.2%
	框架理论	4,849 (8.4%)		+355.8%

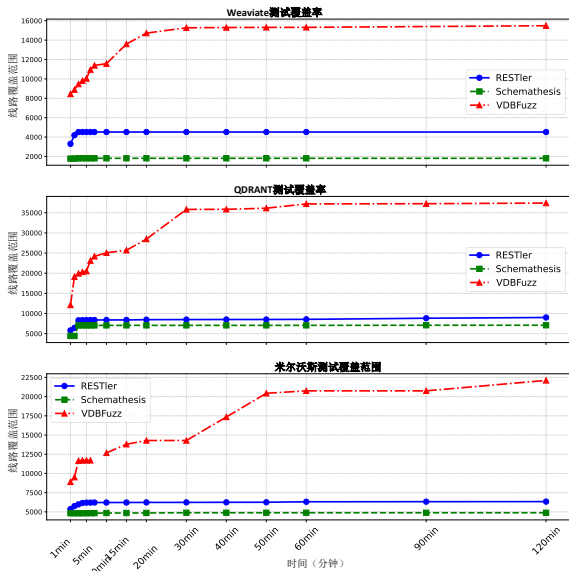


图 3: RESTler、Schemathesis 与 VDBFuzz 在 Weaviate、Qdrant 及 Milvus平台上的代码覆盖率对比。

● **VDBFuzz**: 该工具以全变异模式运行，该模式结合了API序列变异、输入数据变异以及基于模板的索引与查询变异策略。

为评估各工具的有效性，我们记录了随时间推移实现的总线路覆盖率。本文报告了VDBFuzz五次运行及这些基准测试的平均线路覆盖率数据。

对比结果。表9展示了VDBFuzz相较于基准模糊测试工具RESTler和Schemathesis在Weaviate、Qdrant及Milvus平台上的整体测试能力。结果表明，无论是在测试效率还是测试效果方面，VDBFuzz均显著优于基准工具：其平均行覆盖率是RESTler的3倍以上，是Schemathesis的4倍以上。

具体而言，我们在图3中可视化了测试进度随时间的变化情况，以便详细比较三种VDBMS的代码行覆盖率增长情况。在最初10分钟内，VDBFuzz实现的代码行覆盖率显著高于基准工具，充分体现了其在探索API空间方面的卓越效率。例如，在Weaviate平台上，VDBFuzz已覆盖11,571行代码。

表10: 针对Weaviate、Qdrant和Milvus平台突变策略的消融研究。

战略	Weaviate		Qdrant		米尔沃斯	
VDBFuzz	15,485		37,403		22,109	
仅 VDBFuzz 种子文件	14,842	-643	36,342	-1,061	20,140	-1,969
不含索引的 VDBFuzz	15,469	-16	36,493	-910	20,355	-1,754
VDBFuzz（无查询）	15,331	-154	36,550	-853	21,709	-400

VDBFuzz在10分钟内即可完成覆盖，而RESTler需4,519行代码，Schemathesis则需1,811行。在整个120分钟的测试期间，VDBFuzz始终保持优势：在Weaviate上实现15,485行代码全覆盖，在Qdrant上达到37,403行，在Milvus上达22,109行。这些结果表明，在三种VDBMS中，其覆盖率分别比RESTler高出约3.4至4.2倍，比Schemathesis高出4.5至8.5倍。值得注意的是，RESTler和Schemathesis在测试初期覆盖率增长有限——RESTler仅运行几分钟即趋于平稳，Schemathesis的峰值覆盖率也显著较低，凸显了二者在 VDBMS 模糊测试中的局限性；相比之下，VDBFuzz在整个测试过程中持续提升，成功揭示出日益复杂的API交互机制与实现逻辑。

5.4 消融研究（RQ2）

为评估VDBFuzz中不同突变策略的贡献度，我们通过系统性地禁用特定组件并测量其对Weaviate、Qdrant和Milvus三个数据库中的行覆盖率影响，开展了一项消融实验。表10汇总了实验结果，展示了完整版VDBFuzz与其各突变版本所达到的总行覆盖率。结果显示：完整版VDBFuzz在三个VDBMS中均实现最高行覆盖率——Weaviate达15,485行、Qdrant达37,403行、Milvus达22,109行；禁用特定组件则导致覆盖率持续下降，这体现了不同突变策略之间的互补性。下文将分别分析每种策略的具体影响：

- **仅使用种子变异。**在此变体中，VDBFuzz完全依赖初始种子变异。这导致覆盖率下降幅度最大：Weaviate平台减少643行代码，Qdrant平台减少1,061行代码，Milvus平台减少1,969行代码。这些结果凸显了边缘案例变异在揭示深层实现逻辑方面的重要性——而仅使用种子变异的方法无法探索这些内容。
- **索引特异性突变处理。**当禁用索引特异性突变处理功能时，Weaviate平台的测序覆盖度下降了16行，Qdrant平台下降了910行，Milvus平台则下降了1,754行。该策略对Milvus和Qdrant平台的影响尤为显著。
- **查询特定变异。**禁用该策略虽略微降低了数据覆盖率——Weaviate平台减少154行、Qdrant平台减少853行、Milvus平台减少400行。虽然其影响较索引特定变异较小，但对提升查询相关API行为的分析能力至关重要，尤其在Weaviate平台上效果更为显著。

Table 11: VDBFuzz has discovered 19 undisclosed crash bugs and exceptions.

VDBMS	Crash	Exception	Types
Weaviate	2	2	OOBR, Type Assertion Errors
Qdrant	3	1	OOM, FPE, DoS
Milvus	-	2	Query Inconsistencies
Faiss	3	-	OOBR, FPE, NPE
Hnswlib	1	-	DF, OOBR
Annoy	2	-	OOBR
pgvector	-	1	-
sqlite-vec	2	-	OOBR
Total	13	6	-

Overall, the ablation study demonstrates that all components of VDBFuzz contribute meaningfully to its effectiveness. While initial seeds provide a strong baseline, the addition of query-specific and index-specific strategies enables VDBFuzz to systematically and comprehensively explore the API space, achieving significantly higher coverage across diverse VDBMSs.

5.5 Real-world Bugs Discovered (RQ3)

To evaluate the effectiveness of VDBFuzz in uncovering real-world vulnerabilities, we applied it to test 8 VDBMS implementations, including native VDBMSs (Weaviate, Qdrant, and Milvus), VDBMS libraries (Faiss, Hnswlib, and Annoy), and extended VDBMSs (pgvector, sqlite-vec). For the evaluation, VDBFuzz generated and executed more than 10 million valid test cases, which cover strings specific to VDBMSs and particular data fields. VDBFuzz demonstrates the capability to uncover a wide range of security vulnerabilities, such as Out-of-Bound Read (OOBR), Float Point Exception (FPE), Double Free (DF). As illustrated in Table 11, it has successfully identified 13 crash bugs and 6 exception issues, which could potentially result in critical consequences, including sensitive data breaches and the malfunction of LLM services.

```

1 index := createEmptyHnswIndexForTests(t, testVectorForID)
2
3 for i, vec := range testVectors {
4     err := index.Add(uint64(i), vec)
5     require.Nil(t, err)
6 }
7
8 t.Run("searching within cluster 1", func(t *testing.T) {
9     position := 0
10    res, _, err := index.knnSearchByVector(testVectors[
11        position], -1, 36, nil)
12    require.Nil(t, err)
13    assert.ElementsMatch(t, []uint64{0, 1, 2}, res)
14 })

```

Listing 2: Panic in knnSearchByVector with negative k.

Case Study#1 (Weaviate): Panic in knnSearchByVector. One notable vulnerability uncovered by VDBFuzz was an index out-of-bounds crash in Weaviate’s HNSW implementation during a vector search operation, which is shown in Listing 2. The issue occurred when a negative value ($k=-1$) was passed to the knnSearchByVector function (line 10), which controls the number of nearest neighbors to retrieve. The negative value violated the function’s implicit assumption that k would always be positive. This caused a panic

due to an out-of-bounds access. This case highlights the importance of robust boundary checks for critical parameters to prevent such crashes, which could otherwise compromise the reliability and availability of the system.

Case Study#2 (Qdrant): Denial of Service. Another significant vulnerability identified by VDBFuzz was a Denial of Service (DoS) issue in Qdrant. This issue arises when an excessively large vector dimension (dim) is provided as input to the constructor, as shown in Listing 3 (line 7). The function new performs calculations based on the dimension size, such as determining vector_size and chunk_capacity . However, when dim is extremely large (e.g., 2^{63}), the multiplication $\text{dim} * \text{mem}::\text{size_of}::<T>()$ can result in integer overflow, potential leading to divide-by-zero in $\text{chunk_capacity} = \text{CHUNK_SIZE} / \text{vector_size}$. Such a panic results in the service crashing, causing a DoS. For VDBMSs that serve as the components of LLM-based applications, such DoS vulnerabilities are particularly severe, as they can directly render downstream retrieval-augmented generation or semantic search services unavailable.

```

1 // Vulnerable function in ChunkedVectors<T>
2 impl<T: Copy + Clone + Default> ChunkedVectors<T> {
3     pub fn new(dim: usize) -> Self {
4         assert_ne!(dim, 0, "Dimension cannot be 0");
5         let vector_size = dim * mem::size_of::<T>();
6         let chunk_capacity = CHUNK_SIZE / vector_size;
7         assert_ne!(chunk_capacity, 0, "Size is too big");
8     }
9 }
10 // PoC:
11 c = QdrantClient(host="127.0.0.1", port=6333)
12 c.recreate_collection(
13     collection_name="test",
14     vectors_config=models.VectorParams(size=2**63,
15         distance=models.Distance.COSINE),
16 )

```

Listing 3: Vulnerability in ChunkedVectors<T> of Qdrant.

6 DISCUSSION

Limitations. While we implemented VDBFuzz for 8 VDBMSs, our analysis for RQ1 and RQ2 was conducted exclusively on native VDBMS, including Weaviate, Milvus, and Qdrant. These systems were chosen because they expose REST APIs, which provide a direct baseline for comparisons. Additionally, VDBFuzz currently only performs fuzzing for crash-related bugs in vector databases, as crashes present the most direct and observable oracle for identifying potential issues. This excludes deeper considerations of vector search correctness or performance-related bugs, which may also significantly impact real-world applications. Furthermore, VDBFuzz relies on pattern-based mutation on collected seeds, which may not comprehensively cover all API functionalities or edge cases.

Future Work. Future work should explore the development of an oracle for evaluating the correctness of vector search results, as the inherent fuzziness of vector similarity search makes traditional differential testing unsuitable. Another promising direction is the generation of more diverse and comprehensive seed inputs to expand API coverage. LLMs could play a significant role in generating diverse API interactions, given their ability to understand and replicate complex input structures. However, because VDBMS and its APIs evolve rapidly, methods must be devised to ensure that LLMs can stay updated with the latest syntax and functionalities.

表11: VDBFuzz已发现19个未公开的崩溃漏洞及异常情况。

VDBMS	车祸：碰撞	异常：例外	类型
Weaviate	2	2	OOBR，类型断言错误
Qdrant	3	1	OOM, FPE, DoS
米尔沃斯	-	2	查询不一致
法伊斯	3	-	OOBR, FPE, NPE
Hnswlib	1	-	DF, OOBR
烦人：令人不快	2	-	OOBR
pgvector	-	1	-
sqlite-vec	2	-	OOBR
总计	13	6	-

总体而言，这项消融研究表明VDBFuzz的所有组件均对其有效性发挥着重要作用。虽然初始种子提供了坚实的基础，但通过引入查询特定和索引特定的策略，VDBFuzz能够系统且全面地探索API空间，在各类VDBMS中实现显著更高的覆盖率。

5.5 发现的实际世界漏洞（RQ3）

为评估VDBFuzz在发现真实世界安全漏洞方面的有效性，我们将其应用于测试8种 VDBMS 实现方案，包括原生关系型数据库管理系统（Weaviate、Qdrant 和 Milvus）、VDBMS 库（Faiss、Hnswlib 和 Annoy）以及扩展型关系型数据库管理系统（pgvector、sqlite-vec）。在评估过程中，VDBFuzz生成并执行了超过1000万条有效测试用例，这些用例涵盖了关系型数据库管理系统特有的字符串类型及特定数据字段。实验表明，VDBFuzz能够检测多种安全漏洞，如越界读取（OOBR）、浮点异常（FPE）和双重自由指针攻击（DF）。如表11所示，该工具已成功识别出13个导致系统崩溃的错误及6个可能引发严重后果（包括敏感数据泄露和大型语言模型服务故障）的异常情况。

```
1 index := createEmptyHnswIndexForTests(t, testVectorForID)
2
3 for i, vec := range testVectors {
4     err := index.Add(uint64(i), vec)
5     require.Nil(t, err)
6 }
7
8 t.Run("searching within cluster 1", func(t *testing.T) {
9     position := 0
10    res, _, err := index.knnSearchByVector(testVectors[
11        position], -1, 36, nil)
12    require.Nil(t, err)
13    assert.ElementsMatch(t, []uint64{0, 1, 2}, res)
14 })
```

列表 2: 当 k 值为负时，knnSearchByVector 中的恐慌现象。

案例研究#1（Weaviate）：knnSearchByVector函数中的恐慌异常。 VDBFuzz发现的一个显著漏洞是：在执行向量搜索操作时，Weaviate的 HNSW 实现中存在索引越界崩溃问题，具体如列表2所示。该问题发生在向knnSearchByVector函数（第10行）传递负值（k=-1）时——该函数用于控制检索的最近邻数量；该负值违反了函数隐含假设（即k值必须始终为正）。由此引发了系统恐慌异常。

由于超出边界访问所致。该案例凸显了对关键参数进行严格边界验证的重要性，以防止此类崩溃事件发生，否则可能影响系统的可靠性和可用性。

案例研究#2（Qdrant）：服务拒绝攻击。 VDBFuzz发现的另一项重大漏洞是Qdrant中存在的服务拒绝（DoS）问题。该问题发生在向构造函数输入过大的向量维度（dim）时，如代码清单3（第7行）所示。new函数会根据维度大小执行计算（例如确定vector_size和chunk_capacity），但当dim值极大（如263）时，表达式dim * mem::size_of::<T>()可能导致整数溢出，进而引发chunk_capacity = CHUNK_SIZE / vector_size中的除零错误。此类崩溃将导致服务故障，造成DoS攻击。对于作为基于大语言模型应用组件的VDBMS系统而言，此类DoS漏洞尤为严重，因其可能直接使下游的检索增强生成或语义搜索服务无法正常使用。

```
1 // Vulnerable function in ChunkedVectors<T>
2 impl<T: Copy + Clone + Default> ChunkedVectors<T> {
3     pub fn new(dim: usize) -> Self {
4         assert_ne!(dim, 0, "Dimension cannot be 0");
5         let vector_size = dim * mem::size_of::<T>();
6         let chunk_capacity = CHUNK_SIZE / vector_size;
7         assert_ne!(chunk_capacity, 0, "Size is too big");
8     }
9 }
10 // PoC:
11 c = QdrantClient(host="127.0.0.1", port=6333)
12 c.recreate_collection(
13     collection_name="test",
14     vectors_config=models.VectorParams(size=2**63,
15         distance=models.Distance.COSINE),
16 )
```

列表 3: Qdrant 的 ChunkedVectors<T> 中存在的漏洞。

6 讨论

局限性。虽然我们已为8种向量数据库管理系统（VDBMS）实现了VDBFuzz工具，但针对RQ1和RQ2的分析仅在Weaviate、Milvus和Qdrant等原生 VDBMS 平台上进行。选择这些系统是因为它们提供了REST API接口，为对比测试提供了直接基准；此外，VDBFuzz目前仅针对向量数据库中与崩溃相关的漏洞进行模糊测试——因为崩溃现象能最直观地反映潜在问题；这忽略了对向量搜索正确性或性能相关缺陷的深入考量，而这些问题同样可能严重影响实际应用。**未来工作。**未来研究应重点开发用于评估向量搜索结果正确性的验证框架，因为向量相似度搜索固有的模糊性使得传统差异测试方法难以适用。另一个前景广阔的方向是生成更多样化、更全面的种子输入，以扩展API覆盖范围。鉴于大语言模型（LLM）具备理解和复现复杂输入结构的能力，它们在生成多样化的API交互场景方面可发挥重要作用。然而，由于VDBMS及其API发展迅速，必须制定相应方法确保大语言模型能够及时掌握最新的语法规则和功能特性。

7 RELATED WORK

VDBMS Testing. VDBMSs have become more and more critical in the LLM era, yet systematic reliability testing for these systems remains underexplored [58]. Existing works on VDBMS testing have primarily focused on evaluating performance metrics, such as query latency, throughput, and scalability [4, 38, 45, 71]. These studies provide valuable insights but overlook reliability and robustness. The most relevant work MeTMAP introduced metamorphic testing to detect false vector matching problems [26]. However, its focus is on algorithm-level inaccuracies in vector similarity computations rather than implementation-specific defects within VDBMSs. Unlike traditional databases, VDBMSs face distinct challenges due to their reliance on fuzzy semantics in similarity search and hybrid query processing [58, 64]. To bridge this gap, VDBFuzz introduces boundary-specific mutations to discover crash bugs in VDBMS.

DBMS Fuzzing. Fuzz testing has been widely applied to traditional DBMSs to uncover vulnerabilities [41], with existing techniques primarily focusing on crash testing [21, 50], performance testing [27, 30], and logical testing [5, 48, 49]. Additionally, tools tailored for relational databases [5, 57] and NoSQL databases [20, 62, 67] address their respective architectural challenges, such as query correctness, sharding, and flexible schema handling. While these fuzzing techniques have demonstrated effectiveness for traditional databases, they cannot be directly applied to VDBMSs. To this end, VDBFuzz introduces the first fuzz testing framework specifically designed for VDBMSs.

8 CONCLUSION

In this paper, we presented VDBFuzz, the first fuzzing framework specifically designed to detect crash bugs in VDBMSs. We began by conducting an in-depth investigation of crash bugs across state-of-the-art systems and identified boundary condition failures and API sequence dependencies as primary root causes. Based on these insights, VDBFuzz systematically explores crash-inducing behaviors through seed collection, template-based input mutation, and API sequence mutation. Our evaluation on 3 representative VDBMSs, including Weaviate, Milvus, and Qdrant, demonstrated VDBFuzz's effectiveness, achieving up to 3x higher code coverage compared to state-of-the-art fuzzing tools and uncovering 19 previously unknown crash bugs. These findings highlight the potential of VDBFuzz to improve the robustness and reliability of VDBMSs.

Acknowledgement

This work was supported in part by the National Natural Science Foundation of China (grants No.62572209, 62502168) and the Hubei Provincial Key Research and Development Program (grant No. 2025BAB057).

REFERENCES

- [1] activeloopai. 2025. DeepLake: Database for AI. <https://github.com/activeloopai/deeplake>. Accessed: 2025-07-15.
- [2] adriecafe. 2025. Voyager. <https://github.com/adriecafe/voyager>. Accessed: 2025-07-15.
- [3] asg017. 2025. sqlite-vec: A vector search SQLite extension that runs anywhere! <https://github.com/asg017/sqlite-vec>. Accessed: 2025-07-15.
- [4] Martin Aumüller, Erik Bernhardsson, and Alexander John Faithfull. 2020. ANN-Benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Inf. Syst.* 87 (2020). <https://doi.org/10.1016/j.is.2019.02.006>
- [5] Jinsheng Ba and Manuel Rigger. 2023. Testing Database Engines via Query Plan Guidance. In *45th IEEE/ACM International Conference on Software Engineering*, ICSE 2023, Melbourne, Australia, May 14-20, 2023. IEEE, 2060–2071. <https://doi.org/10.1109/ICSE48619.2023.00174>
- [6] Mayank Bawa, Tyson Condie, and Prasanna Ganesan. 2005. LSH forest: self-tuning indexes for similarity search. In *Proceedings of the 14th International Conference on World Wide Web (Chiba, Japan) (WWW '05)*. Association for Computing Machinery, New York, NY, USA, 651–660. <https://doi.org/10.1145/1060745.1060840>
- [7] Alexander Bulekov, Bandan Das, Stefan Hajnoczi, and Manuel Egele. 2023. No Grammar, No Problem: Towards Fuzzing the Linux Kernel without System-Call Descriptions. In *30th Annual Network and Distributed System Security Symposium, NDSS 2023, San Diego, California, USA, February 27 - March 3, 2023*. The Internet Society. <https://www.ndss-symposium.org/ndss-paper/no-grammar-no-problem-towards-fuzzing-the-linux-kernel-without-system-call-descriptions/>
- [8] Shangdong Cao, Ningyu He, Xinyu She, Yixuan Zhang, Mu Zhang, and Haoyu Wang. 2024. WASMaker: Differential Testing of WebAssembly Runtimes via Semantic-Aware Binary Generation. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis, ISSTA 2024, Vienna, Austria, September 16-20, 2024*, Maria Christakis and Michael Pradel (Eds.). ACM, 1262–1273. <https://doi.org/10.1145/3650212.3680358>
- [9] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, and Jianguo Wang. 2024. SingleStore-V: An Integrated Vector Database System in SingleStore. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 3772–3785. <https://doi.org/10.14778/3685800.3685805>
- [10] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaida Jiang, and Zhi Xue. 2022. SFuzz: Slice-based Fuzzing for Real-Time Operating Systems. In *Proceedings of the 2022 ACM SIGSAC Conference on Computer and Communications Security, CCS 2022, Los Angeles, CA, USA, November 7-11, 2022*, Heng Yin, Angelos Stavrou, Cas Cremers, and Elaine Shi (Eds.). ACM, 485–498. <https://doi.org/10.1145/3548606.3559367>
- [11] Weimin Chen, Xiapu Luo, Haipeng Cai, and Haoyu Wang. 2024. Towards Smart Contract Fuzzing on GPUs. In *IEEE Symposium on Security and Privacy, SP 2024, San Francisco, CA, USA, May 19-23, 2024*. IEEE, 2255–2272. <https://doi.org/10.1109/SP54263.2024.00229>
- [12] chroma-core. 2025. Chroma: The AI-native open-source embedding database. <https://github.com/chroma-core/chroma>. Accessed: 2025-07-15.
- [13] Microsoft RESTler Contributors. 2025. RESTler: Stateful REST API Fuzzing Tool. <https://github.com/microsoft/restler-fuzzer>. Accessed: 2025-07-15.
- [14] Schemathesis Contributors. 2025. Schemathesis: Catch API bugs before your users do. <https://github.com/schemathesis/schemathesis>. Version 4.0.9, Accessed: 2025-07-15.
- [15] Facebook Research. 2025. Unit Test of Faiss. <https://github.com/facebookresearch/faiss/tree/main/tests>. Accessed: 2025-07-15.
- [16] facebookresearch. 2025. Faiss: A library for efficient similarity search and clustering of dense vectors. <https://github.com/facebookresearch/faiss>. Accessed: 2025-07-15.
- [17] Faiss. 2024. IndexIVFPQFastScan crashes with certain nlist values. <https://github.com/facebookresearch/faiss/issues/4089>. Accessed: 2025-07-15.
- [18] Faiss. 2025. Faiss indexes. <https://github.com/facebookresearch/faiss/wiki/Faiss-indexes>. Accessed: 2025-07-15.
- [19] Jingzhou Fu, Jie Liang, Zhiyong Wu, and Yu Jiang. 2024. Sedar: Obtaining High-Quality Seeds for DBMS Fuzzing via Cross-DBMS SQL Transfer. In *Proceedings of the 46th IEEE/ACM International Conference on Software Engineering, ICSE 2024, Lisbon, Portugal, April 14-20, 2024*. ACM, 146:1–146:12. <https://doi.org/10.1145/3597503.3639210>
- [20] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang, and Yu Jiang. 2022. Griffin: Grammar-Free DBMS Fuzzing. In *37th IEEE/ACM International Conference on Automated Software Engineering, ASE 2022, Rochester, MI, USA, October 10-14, 2022*. ACM, 49:1–49:12. <https://doi.org/10.1145/3551349.3560431>
- [21] Jingzhou Fu, Jie Liang, Zhiyong Wu, Yanyang Zhao, Shanshan Li, and Yu Jiang. 2025. Understanding and Detecting SQL Function Bugs: Using Simple Boundary Arguments to Trigger Hundreds of DBMS Bugs. In *Proceedings of the Twentieth European Conference on Computer Systems, EuroSys 2025, Rotterdam, The Netherlands, 30 March 2025 - 3 April 2025*. ACM, 1061–1076. <https://doi.org/10.1145/3689031.3696064>
- [22] Xiyue Gao, Zhuang Liu, Jiangtao Cui, Hui Li, Hui Zhang, Kewei Wei, and Kankan Zhao. 2023. A Comprehensive Survey on Database Management System Fuzzing: Techniques, Taxonomy and Experimental Comparison. *CoRR abs/2311.06728* (2023). <https://doi.org/10.48550/ARXIV.2311.06728> arXiv:2311.06728
- [23] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu, and Charles Zhang. 2023. Pinolo: Detecting Logical Bugs in Database Management Systems with Approximate Query Synthesis. In *Proceedings of the 2023 USENIX Annual Technical Conference, USENIX ATC 2023, Boston, MA, USA, July 10-12, 2023*, Julia Lawall and Dan Williams (Eds.). USENIX Association, 345–358. <https://www.usenix.org/conference/atc23/presentation/hao>
- [24] Zihong He, Weizhe Lin, Hao Zheng, Fan Zhang, Matt W. Jones, Laurence Aitchison, Xuhai Xu, Miao Liu, Per Ola Kristensson, and Junxiao Shen. 2024. Human-inspired Perspectives: A Survey on AI Long-term Memory. *CoRR abs/2411.00489*

7 相关工作

VDBMS 测试。在LLM时代, VDBMS系统变得愈发关键, 但针对这些系统的系统性可靠性测试仍缺乏深入研究[58]。现有VDBMS测试研究主要集中在评估查询延迟、吞吐量和可扩展性等性能指标[4, 38, 45, 71], 这些研究虽提供了宝贵见解, 却忽视了系统可靠性和鲁棒性。最具代表性的工作MeTMaP引入了变形测试方法来检测虚假向量匹配问题[26], 但其重点在于向量相似度计算中的算法层面误差, 而非VDBMS系统本身的实现缺陷。与传统数据库不同, VDBMS因依赖模糊语义进行相似度搜索和混合查询处理而面临独特挑战[58, 64]。为弥补这一不足, VDBFuzz通过边界特异性变异来发现VDBMS中的崩溃漏洞。**DBMS模糊测试。**模糊测试已广泛应用于传统DBMS以发现漏洞[41], 现有技术主要聚焦于崩溃测试[21, 50]和性能分析。性能测试[27, 30]与逻辑测试[5, 48, 49]。此外, 专为关系型数据库[5, 57]和NoSQL数据库[20, 62, 67]设计的工具分别针对其特有的架构挑战(如查询正确性、数据分片及灵活模式处理)提供了解决方案。虽然这些模糊测试技术在传统数据库中已展现出有效性, 但无法直接应用于VDBMS。为此, VDBFuzz推出了首个专为VDBMS设计的模糊测试框架。

8 结论

在本文中, 我们推出了VDBFuzz——首个专为检测关系型数据库管理系统(VDBMS)崩溃漏洞而设计的模糊测试框架。我们首先对当前最先进的系统中的崩溃漏洞进行了深入研究, 确定边界条件失效和API调用序列依赖性为主要根源。基于这些发现, VDBFuzz通过种子集收集、模板化输入变异和API调用序列变异等方式, 系统性地探索引发系统崩溃的行为模式。我们在Weaviate、Milvus和Qdrant等三个代表性VDBMS系统上的测试结果充分证明了该框架的有效性: 其代码覆盖率比现有主流模糊测试工具高出三倍, 并发现了19个此前未知的崩溃漏洞。这些成果凸显了VDBFuzz在提升VDBMS系统稳健性和可靠性方面的巨大潜力。

致谢

本研究部分得到了中国国家自然科学基金(项目编号: 62572209、62502168)和湖北省重点研发计划(项目编号: 2025BAB057)的资助。

参考文献

- [1] activeloopai. 2025. DeepLake: 人工智能专用数据库. <https://github.com/activeloopai/deeplake>. 访问日期: 2025-07-15。
- [2] adriecafe. 2025. Voyager. <https://github.com/adriecafe/voyager>. 访问日期: 2025-07-15。
- [3] asg017. 2025. sqlite-vec: 一款可在任何环境中运行的向量搜索SQLite扩展工具! <https://github.com/asg017/sqlite-vec>. 访问日期: 2025-07-15。
- [4] Martin Aumuller, Erik Bernhardtsson 和 Alexander John Faithfull. 2020. ANNBenchmarks: 一种用于近似最近邻算法的基准测试工具. *Inf. Syst.* 87 (2020). <https://doi.org/10.1016/j.is.2019.02.006>
- [5] Jinsheng Ba 和 Manuel Rigger. 2023. 基于查询计划指导的数据库引擎测试. 收录于

- 第45届IEEE/ACM软件工程国际会议, ICSE 2023, 澳大利亚墨尔本, 2023年5月14 - 20日. IEEE, 2060 - 2071. <https://doi.org/10.1109/ICSE48619.2023.00174>
- [6] Mayank Bawa, Tyson Condie 和 Prasanna Ganesan. 2005. LSH Forest: 用于相似性搜索的自调整索引. 收录于第14届万维网国际会议论文集(日本千叶)(WWW'05)。美国纽约州纽约市: 计算机协会, 第651 - 660页. <https://doi.org/10.1145/1060745.1060840>
- [7] Alexander Bulekov, Bandan Das, Stefan Hajnoczi 和 Manuel Egele. 2023. 无语法, 无问题: 无需系统调用描述即可模糊Linux内核. 收录于第30届网络与分布式系统安全年会(NDSS 2023), 美国加利福尼亚州圣地亚哥, 2023年2月27日至3月3日。互联网协会. <https://www.ndss-symposium.org/ndss-paper/no-grammar-no-problem-towards-fuzzing-the-linux-kernel-without-system-call-descriptions/>
- [8] Shangdong Cao, Ningyu He, Xinyu She, Yixuan Zhang, Mu Zhang 和 Haoyu Wang. 2024. WASMaker: 通过语义感知二进制生成对WebAssembly运行时进行差异测试. 收录于第33届ACM sigsoft 国际软件测试与分析研讨会论文集 (ISSTA 2024, 奥地利维也纳, 2024年9月16 - 20日), 玛丽亚·克里斯塔基斯与迈克尔·普拉德尔编. ACM, 第1262 - 1273页. <https://doi.org/10.1145/3650212.3680358>
- [9] Cheng Chen, Chenzhe Jin, Yunan Zhang, Sasha Podolsky, Chun Wu, Szu-Po Wang, Eric Hanson, Zhou Sun, Robert Walzer, 和 Jianguo Wang. 2024. SingleStore-V: SingleStore 中的集成向量数据库系统. *Proc. VLDB Endow.* 17, 12 (2024年8月), 3772 - 3785. <https://doi.org/10.14778/3685800.3685805>
- [10] Libo Chen, Quanpu Cai, Zhenbang Ma, Yanhao Wang, Hong Hu, Minghang Shen, Yue Liu, Shanqing Guo, Haixin Duan, Kaide Jiang, 和 Zhi Xue. 2022. SFuzz: 面向实时操作系统的切片模糊测试. 收录于《2022年ACM SIGSAC 计算机与通信安全会议论文集(中国化学会, 美国加利福尼亚州洛杉矶, 2022年11月7 - 11日)》, 作者: 殷恒、Angelos Stavrou、Cas Cremers 和 Elaine Shi (编)。ACM, 第485 - 498页. <https://doi.org/10.1145/3548606.3559367>
- [11] Weimin Chen, Xiapu Luo, Haipeng Cai 和 Haoyu Wang. 2024. 基于 GPU 的智能合约模糊测试研究. 收录于《IEEE 安全与隐私研讨会 (SP 2024)》, 美国加利福尼亚州旧金山, 2024年5月19 - 23日. IEEE, 2255 - 2272. <https://doi.org/10.1109/SP54263.2024.00229>
- [12] Chroma-Core. 2025年. Chroma: 一款专为人工智能设计的开源嵌入数据库. <https://github.com/chroma-core/chroma>. 访问日期: 2025-07-15。
- [13] 微软 RESTler 贡献者. 2025年. RESTler: 状态化 REST API 漏洞测试工具. <https://github.com/microsoft/restler-fuzzer>. 访问日期: 2025-07-15。
- [14] Schemathesis 贡献者. 2025年. Schemathesis: 在用户发现之前捕捉 API 错误. <https://github.com/schemathesis/schemathesis>. 版本 4.0.9, 访问日期: 2025-07-15。
- [15] Facebook Research. 2025. Faiss 的单元测试. <https://github.com/facebookresearch/faiss/tree/main/tests>. 访问日期: 2025-07-15。
- [16] facebookresearch. 2025. Faiss: 一个用于高效相似性搜索及密集向量聚类的库. <https://github.com/facebookresearch/faiss>. 访问日期: 2025-07-15。
- [17] Faiss. 2024. IndexIVFPQFastScan 在某些 nlist 值下会出现崩溃. <https://github.com/facebookresearch/faiss/issues/4089>. 访问日期: 2025-07-15。
- [18] Faiss. 2025. Faiss 索引. <https://github.com/facebookresearch/faiss/wiki/Faiss索引>. 访问日期: 2025-07-15。
- [19] Jingzhou Fu, Jie Liang, Zhiyong Wu 和 Yu Jiang. 2024. Sedar: 通过跨数据库管理系统 SQL 数据传输获取高质量的数据库管理系统模糊测试种子. 收录于《第46届IEEE/ACM国际软件工程会议论文集, ICSE 2024, 葡萄牙里斯本, 2024年4月14 - 20日》. ACM, 146:1 - 146:12. <https://doi.org/10.1145/3597503.3639210>
- [20] Jingzhou Fu, Jie Liang, Zhiyong Wu, Mingzhe Wang 和 Yu Jiang. 2022. Griffin: 无需语法的数据库管理系统模糊测试. 收录于第37届IEEE/ACM自动化软件工程国际会议 (ASE 2022), 美国密歇根州罗切斯特市, 2022年10月10 - 14日. ACM, 49:1 - 49:12. <https://doi.org/10.1145/3551349.3560431>
- [21] Jingzhou Fu, Jie Liang, Zhiyong Wu, Yanyang Zhao, Shanshan Li 和 Yu Jiang. 2025. 了解与检测 SQL 函数缺陷: 利用简单边界参数触发数百种数据库管理系统缺陷. 收录于第二十届欧洲计算机系统会议论文集 (EuroSys 2025), 荷兰鹿特丹, 2025年3月30日至4月3日. ACM出版社, 第1061 - 1076页. <https://doi.org/10.1145/3689031.3696064>
- [22] Xiyue Gao, Zhuang Liu, Jiangtao Cui, Hui Li, Hui Zhang, Kewei Wei 和 Kankan Zhao. 2023. 数据库管理系统模糊测试的全面综述: 技术方法、分类体系与实验比较. *CoRR abs/2311.06728* (2023). <https://doi.org/10.48550/ARXIV.2311.06728> arXiv: 2311.06728
- [23] Zongyin Hao, Quanfeng Huang, Chengpeng Wang, Jianfeng Wang, Yushan Zhang, Rongxin Wu 和 Charles Zhang. 2023. Pinolo: 利用近似查询综合技术检测数据库管理系统中的逻辑错误. 收录于《2023年USENIX年度技术会议论文集 (USENIX ATC 2023)》, 美国马萨诸塞州波士顿, 2023年7月10 - 12日 (朱莉娅·劳尔与丹·威廉姆斯编), USENIX 协会, 第345 - 358页. <https://www.usenix.org/conference/atc23/presentation/hao>
- [24] Zihong He, Weizhe Lin, Hao Zheng, Fan Zhang, Matt W. Jones, Laurence Aitchison, Xuhai Xu, Miao Liu, Per Ola Kristensson 和 Junxiao Shen. 2024. 受人类启发的视角: 关于人工智能长期记忆的综述. *CoRR abs/2411.00489* (2024). <https://doi.org/10.48550/ARXIV.2411.00489>

- (2024). <https://doi.org/10.48550/ARXIV.2411.00489> arXiv:2411.00489
- [25] Xun Jiang, Feng Li, Han Zhao, Jiaying Wang, Jun Shao, Shihao Xu, Shu Zhang, Weiling Chen, Xavier Tang, Yize Chen, Mengyue Wu, Weizhi Ma, Mengdi Wang, and Tianqiao Chen. 2024. Long Term Memory: The Foundation of AI Self-Evolution. *CoRR* abs/2410.15665 (2024). <https://doi.org/10.48550/ARXIV.2410.15665> arXiv:2410.15665
- [26] Zhi Jing, Yongye Su, Yikun Han, Bo Yuan, Haiyun Xu, Chunjiang Liu, Kehai Chen, and Min Zhang. 2024. When Large Language Models Meet Vector Databases: A Survey. *CoRR* abs/2402.01763 (2024). <https://doi.org/10.48550/ARXIV.2402.01763> arXiv:2402.01763
- [27] Jinho Jung, Hong Hu, Joy Arulraj, Taesoo Kim, and Woon-Hak Kang. 2019. APOLLO: Automatic Detection and Diagnosis of Performance Regressions in Database Systems. *Proc. VLDB Endow.* 13, 1 (2019), 57–70. <https://doi.org/10.14778/3357377.3357382>
- [28] Herve Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. <https://doi.org/10.1109/TPAMI.2010.57>
- [29] lancedb. 2025. LanceDB: The Multimodal AI Lakehouse. <https://github.com/lancedb/lancedb>. Accessed: 2025-07-15.
- [30] Xinyu Liu, Qi Zhou, Joy Arulraj, and Alessandro Orso. 2022. Automatic Detection of Performance Bugs in Database Systems using Equivalent Queries. In *44th IEEE/ACM 44th International Conference on Software Engineering, ICSE 2022, Pittsburgh, PA, USA, May 25-27, 2022*. ACM, 225–236. <https://doi.org/10.1145/3510003.3510093>
- [31] Yingfan Liu, Jiangtao Cui, Zi Huang, Hui Li, and Heng Tao Shen. 2014. SK-LSH: an efficient index structure for approximate nearest neighbor search. *Proc. VLDB Endow.* 7, 9 (May 2014), 745–756. <https://doi.org/10.14778/2732939.2732947>
- [32] Yu A. Malkov and D. A. Yashunin. 2020. Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42, 4 (2020), 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>
- [33] marqo ai. 2025. marqo: Unified embedding generation and search engine. <https://github.com/marqo-ai/marqo>. Accessed: 2025-07-15.
- [34] Milvus. 2025. Integration Test of Milvus. <https://github.com/milvus-io/milvus/tree/master/tests/integration>. Accessed: 2025-07-15.
- [35] milvus-io. 2025. Milvus: High-performance, cloud-native vector database built for scalable vector ANN search. <https://github.com/milvus-io/milvus>. Accessed: 2025-07-15.
- [36] MongoDB. 2025. MongoDB Atlas Vector Search. <https://www.mongodb.com/products/platform/atlas-vector-search>. Accessed: 2025-07-15.
- [37] Mozilla. 2025. grcov: Rust tool to collect and aggregate code coverage data for multiple source files. <https://github.com/mozilla/grcov> Version 0.10.0, Accessed: 2025-07-15.
- [38] MyScale. 2023. MyScale Vector Database Benchmark. <https://myscale.github.io/benchmark/>. Accessed: 2025-07-15.
- [39] neuml. 2025. txtai: All-in-one open-source AI framework for semantic search, LLM orchestration and language model workflows. <https://github.com/neuml/txtai>. Accessed: 2025-07-15.
- [40] nmslib. 2025. Hnswlib: Header-only C++/python library for fast approximate nearest neighbors. <https://github.com/nmslib/hnswlib>. Accessed: 2025-07-15.
- [41] James Jie Pan, Jianguo Wang, and Guoliang Li. 2024. Survey of vector database management systems. *VLDB J.* 33, 5 (2024), 1591–1615. <https://doi.org/10.1007/S00778-024-00864-X>
- [42] pgvector. 2025. pgvector: Open-source vector similarity search for Postgres. <https://github.com/pgvector/pgvector>. Accessed: 2025-07-15.
- [43] Pinecone. 2025. Pinecone: The vector database to build knowledgeable AI. <https://www.pinecone.io/>. Accessed: 2025-07-15.
- [44] PostgreSQL. 2025. PostgreSQL: The World's Most Advanced Open Source Relational Database. <https://www.postgresql.org/>. Accessed: 2025-07-15.
- [45] Qdrant. 2024. Vector Database Benchmarks. <https://qdrant.tech/benchmarks/>. Accessed: 2025-07-15.
- [46] Qdrant. 2025. Qdrant: High-performance, massive-scale vector database and vector search engine for the next generation of AI. <https://github.com/qdrant/qdrant>. Accessed: 2025-07-15.
- [47] Qiniu. 2023. goc: A Comprehensive Coverage Testing System for The Go Programming Language. <https://github.com/qiniu/goc> Version 1.4.5, Accessed: 2025-07-15.
- [48] Manuel Rigger and Zhendong Su. 2020. Detecting optimization bugs in database engines via non-optimizing reference engine construction. In *ESEC/FSE '20: 28th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering, Virtual Event, USA, November 8-13, 2020*, Prem Devanbu, Myra B. Cohen, and Thomas Zimmermann (Eds.). ACM, 1140–1152. <https://doi.org/10.1145/3368089.3409710>
- [49] Manuel Rigger and Zhendong Su. 2020. Testing Database Engines via Pivoted Query Synthesis. In *14th USENIX Symposium on Operating Systems Design and Implementation, OSDI 2020, Virtual Event, November 4-6, 2020*. USENIX Association, 667–682. <https://www.usenix.org/conference/osdi20/presentation/rigger>
- [50] Andreas Seltenreich, Bo Tang, and Sjoerd Mullender. 2022. SQLsmith: A random SQL query generator. <https://github.com/anse1/sqlsmith>. <https://github.com/anse1/sqlsmith> Version 1.4.
- [51] spotify. 2024. annoy. <https://github.com/spotify/annoy>. Accessed: 2025-07-15.
- [52] sqlite. 2025. sqlite: Official Git mirror of the SQLite source tree. <https://github.com/sqlite/sqlite>. Accessed: 2025-07-15.
- [53] Toni Taipalus. 2024. Vector database management systems: Fundamental concepts, use-cases, and current challenges. *Cogn. Syst. Res.* 85 (2024), 101216. <https://doi.org/10.1016/J.COGLYS.2024.101216>
- [54] unum cloud. 2025. USearch. <https://github.com/unum-cloud/usearch>. Accessed: 2025-07-15.
- [55] vespa engine. 2025. Vespa. <https://github.com/vespa-engine/vespa>. Accessed: 2025-07-15.
- [56] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xianguan Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: A Purpose-Built Vector Data Management System. In *Proceedings of the 2021 International Conference on Management of Data (Virtual Event, China) (SIGMOD '21)*. Association for Computing Machinery, New York, NY, USA, 2614–2627. <https://doi.org/10.1145/3448016.3457550>
- [57] Mingzhe Wang, Zhiyong Wu, Xinyi Xu, Jie Liang, Chijin Zhou, Huafeng Zhang, and Yu Jiang. 2021. Industry Practice of Coverage-Guided Enterprise-Level DBMS Fuzzing. In *43rd IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, ICSE (SEIP) 2021, Madrid, Spain, May 25-28, 2021*. IEEE, 328–337. <https://doi.org/10.1109/ICSE-SEIP52600.2021.00042>
- [58] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou, and Haoyu Wang. 2025. Towards Reliable Vector Database Management Systems: A Software Testing Roadmap for 2030. *CoRR* abs/2502.20812 (2025). <https://doi.org/10.48550/ARXIV.2502.20812> arXiv:2502.20812
- [59] Weaviate. 2024. Uncontrolled concurrency in batch delete may crash server. <https://github.com/weaviate/weaviate/issues/5093>. Accessed: 2025-07-15.
- [60] Weaviate. 2025. Unit Test of Weaviate. <https://github.com/weaviate/weaviate/tree/main/test>. Accessed: 2025-07-15.
- [61] Weaviate. 2025. Weaviate: Open-source vector database that stores both objects and vectors. <https://github.com/weaviate/weaviate>. Accessed: 2025-07-15.
- [62] Zhiyong Wu, Jie Liang, Mingzhe Wang, Chijin Zhou, and Yu Jiang. 2022. Unicorn: detect runtime errors in time-series databases with hybrid input synthesis. In *ISSTA '22: 31st ACM SIGSOFT International Symposium on Software Testing and Analysis, Virtual Event, South Korea, July 18 - 22, 2022*, Sukyoung Ryu and Yannis Smaragdakis (Eds.). ACM, 251–262. <https://doi.org/10.1145/3533767.3534364>
- [63] Xingrui Xie, Han Liu, Wenzhe Hou, and Hongbin Huang. 2023. A Brief Survey of Vector Databases. In *2023 9th International Conference on Big Data and Information Analytics (BigDIA)*. 364–371. <https://doi.org/10.1109/BigDIA60676.2023.10429609>
- [64] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Shenao Wang, Kai Chen, and Haoyu Wang. 2025. Toward Understanding Bugs in Vector Database Management Systems. *CoRR* abs/2506.02617 (2025). <https://doi.org/10.48550/ARXIV.2506.02617> arXiv:2506.02617
- [65] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu, and Haoyu Wang. 2025. CKGFuzzer: LLM-Based Fuzz Driver Generation Enhanced By Code Knowledge Graph. In *4th IEEE/ACM International Conference on Software Engineering, ICSE 2025 - Companion Proceedings, Ottawa, ON, Canada, April 27 - May 3, 2025*. IEEE, 243–254. <https://doi.org/10.1109/ICSE-COMPANION66252.2025.00079>
- [66] Wen Yang, Tao Li, Gai Fang, and Hong Wei. 2020. PASE: PostgreSQL Ultra-High-Dimensional Approximate Nearest Neighbor Search Extension. In *Proceedings of the 2020 ACM SIGMOD International Conference on Management of Data (Portland, OR, USA) (SIGMOD '20)*. Association for Computing Machinery, New York, NY, USA, 2241–2253. <https://doi.org/10.1145/3318464.3386131>
- [67] Yupeng Yang, Yongheng Chen, Rui Zhong, Jizhou Chen, and Wenke Lee. 2024. Towards Generic Database Management System Fuzzing. In *33rd USENIX Security Symposium, USENIX Security 2024, Philadelphia, PA, USA, August 14-16, 2024*, Davide Balzarotti and Wenyuan Xu (Eds.). USENIX Association. <https://www.usenix.org/conference/usenixsecurity24/presentation/yang-yupeng>
- [68] Qiang Zhang, Yuheng Shen, Jianzhong Liu, Yiru Xu, Heyuan Shi, Yu Jiang, and Wanli Chang. 2024. ECG: Augmenting Embedded Operating System Fuzzing via LLM-Based Corpus Generation. *IEEE Trans. Comput. Aided Des. Integr. Circuits Syst.* 43, 11 (2024), 4238–4249. <https://doi.org/10.1109/TCAD.2024.3447220>
- [69] Yunan Zhang, Shige Liu, and Jianguo Wang. 2024. Are There Fundamental Limitations in Supporting Vector Data Management in Relational Databases? A Case Study of PostgreSQL. In *2024 IEEE 40th International Conference on Data Engineering (ICDE)*. 3640–3653. <https://doi.org/10.1109/ICDE60146.2024.00280>
- [70] Xinyang Zhao, Xuanhe Zhou, and Guoliang Li. 2024. Chat2Data: An Interactive Data Analysis System with RAG, Vector Databases and LLMs. *Proc. VLDB Endow.* 17, 12 (Aug. 2024), 4481–4484. <https://doi.org/10.14778/3685800.3685905>
- [71] zilliztech. 2025. VectorDBBench: A Vector Database Benchmark Tool. <https://zilliz.com/benchmark>. Accessed: 2025-07-15.

- [org/10.48550/ARXIV.2411.00489](https://arxiv.org/abs/2411.00489) arXiv:2411.00489
- [25] Xun Jiang, Feng Li, Han Zhao, Jiaying Wang, Jun Shao, Shihao Xu, Shu Zhang, Weiling Chen, Xavier Tang, Yize Chen, Mengyue Wu, Weizhi Ma, Mengdi Wang, and Tianqiao Chen. 2024. 长期记忆: 人工智能自我演化的基础. *CoRR* abs/2410.15665 (2024). <https://doi.org/10.48550/ARXIV.2410.15665> arXiv:2410.15665
 - [26] Zhi Jing, Yongye Su, Yikun Han, Bo Yuan, Haiyun Xu, Chunjiang Liu, Kehai Chen and Min Zhang. 2024. 大型语言模型与向量数据库的结合: 综述. *CoRR* abs/2402.01763 (2024). <https://doi.org/10.48550/ARXIV.2402.01763> arXiv: 2402.01763
 - [27] Jinho Jung, Hong Hu, Joy Arulraj, Taesoo Kim and Woon-Hak Kang. 2019. Apollo: 数据库系统中性能退化的自动检测与诊断. *Vldb 基金会会议录* 13卷第1期 (2019年), 第57–70页. <https://doi.org/10.14778/3357377.3357382>
 - [28] Herve Jegou, Matthijs Douze and Cordelia Schmid. 2011. 最邻近搜索中的产品量化方法. *IEEE 模式分析与机器学习汇刊* 33卷第1期 (2011年), 第117–128页. <https://doi.org/10.1109/TPAMI.2010.57>
 - [29] lancedb. 2025. LanceDB: 多模态 AI 数据仓库. <https://github.com/lancedb/lancedb>. 访问日期: 2025-07-15.
 - [30] Xinyu Liu, Qi Zhou, Joy Arulraj and Alessandro Orso. 2022. 使用等效查询自动检测数据库系统中的性能缺陷. 收录于 *第44届IEEE/ACM国际软件工程大会 (ICSE 2022)*, 美国宾夕法尼亚州匹兹堡, 2022年5月25–27日. ACM, 225–236. <https://doi.org/10.1145/3510003.3510093>
 - [31] Yingfan Liu, Jiangtao Cai, Zi Huang, Hui Li and Heng Tao Shen. 2014. SK-LSH: 一种用于近似最近邻搜索的高效索引结构. *Vldb Endow 会议论文集*, 7,9 (2014年5月), 745–756. <https://doi.org/10.14778/2732939.2732947>
 - [32] Yu A. Malkov and D. A. Yul'f. 2020年. 基于层次可导航小世界图的高效稳健近似最近邻搜索. *IEEE 模式分析与机器学习汇刊* 第42卷第4期 (2020年), 第824–836页. <https://doi.org/10.1109/TPAMI.2018.2889473>
 - [33] Marqo AI. 2025. Marqo: 统一嵌入生成与搜索引擎. <https://github.com/marqo-ai/marqo>. 访问日期: 2025-07-15.
 - [34] Milvus. 2025. Milvus 集成测试. <https://github.com/milvus-io/milvus/tree/master/tests/integration>. 访问日期: 2025-07-15.
 - [35] milvus-io. 2025. Milvus: 专为可扩展向量人工智能搜索设计的高性能、原生云向量数据库. <https://github.com/milvus-io/milvus>. 访问日期: 2025-07-15.
 - [36] MongoDB. 2025. MongoDB Atlas 矢量搜索. <https://www.mongodb.com/products/platform/atlas-vector-search>. 访问日期: 2025-07-15.
 - [37] Mozilla. 2025. grcov: 一款用于收集和汇总多个源代码覆盖率数据的 Rust 工具. <https://github.com/mozilla/grcov> 版本 0.10.0, 访问日期: 2025-07-15.
 - [38] MyScale. 2023. MyScale 向量数据库基准测试. <https://myscale.github.io/benchmark/>. 访问日期: 2025年7月15日.
 - [39] neuml. 2025. txtai: 一款集成了语义搜索、大语言模型编排及语言模型工作流程功能的一体化开源 AI 框架. <https://github.com/neuml/txtai>. 访问日期: 2025-07-15.
 - [40] nmslib. 2025. hnswlib: 一个仅包含头文件的 C++/Python 库, 用于快速计算近似最近邻关系. <https://github.com/nmslib/hnswlib>. 访问日期: 2025-07-15.
 - [41] 潘杰、王建国和李国亮. 2024年. 向量数据库管理系统综述. *Vldb J* 33,5 (2024), 1591–1615. <https://doi.org/10.1007/S00778-024-00864-X>
 - [42] pgvector. 2025. pgvector: 面向 Postgres 的开源向量相似性搜索工具. <https://github.com/pgvector/pgvector>. 访问日期: 2025-07-15.
 - [43] Pinecone. 2025. Pinecone: 构建知识型人工智能的向量数据库. <https://www.pinecone.io/>. 访问日期: 2025-07-15.
 - [44] PostgreSQL. 2025. PostgreSQL: 全球最先进的开源关系型数据库. <https://www.postgresql.org/>. 访问日期: 2025年7月15日.
 - [45] Qdrant. 2024. 向量数据库基准测试. <https://qdrant.tech/benchmarks/>. 访问日期: 2025-07-15.
 - [46] Qdrant. 2025年. Qdrant: 面向下一代人工智能的高性能、大规模向量数据库及向量搜索引擎. <https://github.com/qdrant/qdrant>. 访问日期: 2025-07-15.
 - [47] Qiniu. 2023. goc: 一种面向 Go 编程语言的全面覆盖率测试系统. <https://github.com/qiniu/goc> 版本 1.4.5, 访问日期: 2025-07-15.
 - [48] Manuel Rigger and Zhendong Su. 2020. 通过构建非优化参考引擎检测数据库引擎中的优化错误. 收录于 *《ESEC/FSE '20: 第28届ACM欧洲联合软件工程会议暨软件工程基础研讨会 (虚拟活动)》*, 美国, 2020年11月8–13日, 由 Prem Devanbu, Myra B. Cohen 和 Thomas Zimmermann 编. ACM 出版社, 第1140–1152页. <https://doi.org/10.1145/3368089.3409710>
 - [49] 曼努埃尔·里格与苏振东. 2020年. 通过枢轴查询合成测试数据库引擎. 收录于 *第14届USENIX 操作系统设计与实现研讨会, OSDI 2020 虚拟会议, 2020年11月4–6日*. USENIX 协会, 第667–682页. <https://www.usenix.org/conference/osdi20/presentation/rigger>.
 - [50] Andreas Seltenreich, Bo Tang and Sjoerd Mullender. 2022. SQLSmith: 随机SQL查询生成器. <https://github.com/anse1/sqlsmith>. <https://github.com/anse1/sqlsmith> 版本 1.4.
 - [51] spotify. 2024. annoy. <https://github.com/spotify/annoy>. 访问日期: 2025-07-15.
 - [52] sqlite. 2025. sqlite: SQLite 源代码树的官方 Git 镜像. <https://github.com/sqlite/sqlite>. 访问日期: 2025-07-15.
 - [53] 托尼·泰普拉斯. 2024. 向量数据库管理系统: 基本概念、应用场景及当前挑战. *Cogn. Syst. Res.* 85 (2024), 101216. <https://doi.org/10.1016/j.cogsys.2024.101216>
 - [54] Unum Cloud. 2025年. USearch. <https://github.com/unum-cloud/usearch>. 访问日期: 2025-07-15.
 - [55] Vespa 发动机. 2025年. Vespa. <https://github.com/vespa-engine/vespa>. 访问日期: 2025-07-15.
 - [56] Jianguo Wang, Xiaomeng Yi, Rentong Guo, Hai Jin, Peng Xu, Shengjun Li, Xiangyu Wang, Xiangzhou Guo, Chengming Li, Xiaohai Xu, Kun Yu, Yuxing Yuan, Yinghao Zou, Jiquan Long, Yudong Cai, Zhenxiang Li, Zhifeng Zhang, Yihua Mo, Jun Gu, Ruiyi Jiang, Yi Wei, and Charles Xie. 2021. Milvus: 一款专为特定用途设计的向量数据库管理系统. 收录于 *《2021年国际数据管理会议论文集》* (虚拟会议, 中国) (SIGMOD '21). 计算机协会, 美国纽约州纽约市, 第2614–2627页. <https://doi.org/10.1145/3448016.3457550>
 - [57] Mingzhe Wang, Zhiyong Wu, Xinyi Xu, Jie Liang, Chijin Zhou, Huafeng Zhang, and Yu Jiang. 2021. 覆盖度引导的企业级数据库管理系统模糊测试的行业实践. 收录于 *第43届IEEE/ACM国际软件工程会议: 软件工程实践, ICSE (SEIP) 2021*, 西班牙马德里, 2021年5月25–28日. IEEE, 328–337. <https://doi.org/10.1109/ICSE-SEIP52600.2021.00042>
 - [58] Shenao Wang, Yanjie Zhao, Yinglin Xie, Zhao Liu, Xinyi Hou, Quanchen Zou and Haoyu Wang. 2025. 建立可靠的向量数据库管理系统: 面向2030年的软件测试路线图. *CoRR* abs/2502.20812 (2025). <https://doi.org/10.48550/ARXIV.2502.20812> arXiv:2502.20812
 - [59] Weaviate. 2024年. 批量删除操作中的非受控并发可能导致服务器崩溃. <https://github.com/weaviate/weaviate/issues/5093>. 访问日期: 2025-07-15.
 - [60] Weaviate. 2025年. Weaviate 单元测试. <https://github.com/weaviate/weaviate/tree/main/test>. 访问日期: 2025-07-15.
 - [61] Weaviate. 2025年. Weaviate: 一个开源矢量数据库, 可同时存储对象和矢量. <https://github.com/weaviate/weaviate>. 访问日期: 2025-07-15.
 - [62] Zhiyong Wu, Jie Liang, Mingzhe Wang, Chijin Zhou and Yu Jiang. 2022. 《Unicorn: 利用混合输入合成技术检测时间序列数据库中的运行时错误》. 收录于 *《ISSTA '22: 第31届ACM SigSoft 国际软件测试与分析研讨会 (虚拟会议)》*, 韩国, 2022年7月18–22日, 由 Sukyoung Ryu 和 Yannis Smaragdakis 编. ACM, 第251–262页. <https://doi.org/10.1145/3533767.3534364>
 - [63] Xingrui Xie, Han Liu, Wenzhe Hou and Hongbin Huang. 2023. 向量数据库简要综述. 收录于 *2023年第9届大数据与信息分析国际会议 (BigDIA)*. 364–371. <https://doi.org/10.1109/BigDIA60676.2023.10429609>
 - [64] Yinglin Xie, Xinyi Hou, Yanjie Zhao, Shenao Wang, Kai Chen and Haoyu Wang. 2025. 探讨向量数据库管理系统中的漏洞问题. *CoRR* abs/2506.02617 (2025). <https://doi.org/10.48550/ARXIV.2506.02617> arXiv: 2506.02617
 - [65] Hanxiang Xu, Wei Ma, Ting Zhou, Yanjie Zhao, Kai Chen, Qiang Hu, Yang Liu and Haoyu Wang. 2025. CKGFuzzer: 基于代码知识图谱增强的 LLM 驱动程序生成工具. 收录于 *第47届IEEE/ACM国际软件工程会议, ICSE 2025——配套论文集, 加拿大 安大略省渥太华, 2025年4月27日至5月3日*. IEEE, 243–254. <https://doi.org/10.1109/ICSE-COMPANION66252.2025.00079>
 - [66] Wen Yang, Tao Li, Gai Fang and Hong Wei. 2020. PASE: PostgreSQL 超高维近似最近邻搜索扩展. 收录于 *《2020年ACM SIGMOD 国际数据管理会议论文集》* (美国俄勒冈州波特兰市) (SIGMOD '20). 美国计算机协会, 纽约州纽约市, 第2241–2253页. <https://doi.org/10.1145/3318464.3386131>
 - [67] Yupeng Yang, Yongheng Chen, Rui Zhong, Jizhou Chen and Wenke Lee. 2024. 面向通用数据库管理系统的模糊测试研究. 收录于 *第33届USENIX 安全研讨会 (USENIX Security 2024)*, 美国宾夕法尼亚州费城, 2024年8月14–16日, 由 Davide Balzarotti 和 Wenyuan Xu 主编. USENIX 协会. <https://www.usenix.org/conference/usenixsecurity24/presentation/yang-yupeng>
 - [68] Qiang Zhang, Yuheng Shen, Jianzhong Liu, Yiru Xu, Heyuan Shi, Yu Jiang and Wanli Chang. 2024. 心电图: 基于大语言模型语料库生成增强嵌入式操作系统模糊测试. *IEEE Transactions on Computer-Aided Design and Integrated Circuits Systems* 43,11 (2024), 4238–4249. <https://doi.org/10.1109/TCAD.2024.3447220>
 - [69] Yunan Zhang, Shige Liu and Jianguo Wang. 2024. 关系型数据库中支持向量数据管理是否存在根本性局限? 以 PostgreSQL 为例. 收录于 *2024 年 IEEE 第 40 届国际数据工程会议 (ICDE)*. 3640–3653. <https://doi.org/10.1109/ICDE60146.2024.00280>
 - [70] Xinyang Zhao, Xuanhe Zhou and Guoliang Li. 2024. Chat2Data: 一个结合 RAG、向量数据库和大语言模型的交互式数据分析系统. *Vldb 基金会会议录* 17,12 (2024年8月), 4481–4484. <https://doi.org/10.14778/3685800.3685905>
 - [71] zilliztech. 2025. VectorDBBench: 一种向量数据库基准测试工具. <https://zilliz.com/benchmark>. 访问日期: 2025-07-15.